

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NGUYỄN VIỆT TÙNG

NGUYỄN LỘC TỶ

ĐỒ ÁN CHUYÊN NGÀNH
NGHIÊN CỨU PHÁT HIỆN TẤN CÔNG DENIAL OF
WALLET BẰNG TRÍ TUỆ NHÂN TẠO
ARTIFICIAL INTELLIGENCE-BASED DETECTION FOR
DENIAL OF WALLET

NGÀNH AN TOÀN THÔNG TIN

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NGUYỄN VIỆT TÙNG - 23521746

NGUYỄN LỘC TỶ - 23521757

ĐỒ ÁN CHUYÊN NGÀNH
NGHIÊN CỨU PHÁT HIỆN TẤN CÔNG DENIAL OF
WALLET BẰNG TRÍ TUỆ NHÂN TẠO
ARTIFICIAL INTELLIGENCE-BASED DETECTION FOR
DENIAL OF WALLET

NGÀNH AN TOÀN THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
TS. NGUYỄN NGỌC TỰ

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Chúng em muốn gửi lời cảm ơn chân thành sâu sắc tới giảng viên hướng dẫn TS. Nguyễn Ngọc Tự đã tận tình hỗ trợ nhóm để hoàn thành đồ án này. Qua đồ án này, chúng em đã tích lũy được nhiều kiến thức bổ ích về chuyên môn cũng như rèn luyện kỹ năng làm việc nhóm. Mặc dù các thành viên trong nhóm đã cố gắng hoàn thành đồ án một cách hoàn thiện nhất nhưng do thiếu sót về kinh nghiệm cũng như một số hạn chế khác nên đồ án này cũng không thể tránh khỏi các sai sót. Chúng em rất mong nhận được sự cảm thông, chia sẻ và góp ý từ quý thầy cô cũng như các bạn sinh viên. Chúng em xin chân thành cảm ơn!

MỤC LỤC

Chương 1.	GIỚI THIỆU ĐỀ TÀI	2
1.1.	Giới thiệu về Serverless Computing	2
1.1.1.	Function-as-a-Service	2
1.1.2.	Backend-as-a-Service	2
1.1.3.	Luồng hoạt động cơ bản của Serverless	3
1.1.4.	Các đặc tính cốt lõi của Serverless Computing	3
1.1.5.	Xu hướng chuyển dịch sang kiến trúc Serverless	4
1.2.	Mô hình chia sẻ trách nhiệm trong kiến trúc Serverless	5
1.3.	Thách thức bảo mật trong kiến trúc Serverless	6
1.4.	Cuộc tấn công Denial of Wallet	6
1.5.	Mục tiêu	7
Chương 2.	CÁC NGHIÊN CỨU LIÊN QUAN	8
2.1.	Tổng quan về tấn công Denial of Wallet trong môi trường Serverless	8
2.2.	Phân loại tấn công DoW trong môi trường Serverless	8
2.3.	DoWTS - Bộ sinh dữ liệu phục vụ nghiên cứu về DoW	9
2.4.	Xây dựng bộ dữ liệu chuyên biệt về DoW	10
2.5.	Mở rộng bộ dữ liệu về DoW bằng Generative AI và GAN	12
2.6.	Đóng góp của nhóm	13
Chương 3.	KIẾN TRÚC ĐỀ XUẤT	15
3.1.	Mục tiêu thiết kế kiến trúc	15
3.2.	Kiến trúc tổng quan	16
3.3.	Luồng hoạt động của hệ thống	17
3.4.	Ảnh xạ đặc trưng vào môi trường triển khai	18

3.5. Các vấn đề khi vận hành thực tế	19
Chương 4. HUẤN LUYỆN MÔ HÌNH	21
4.1. Tổng quan về Dataset (Dữ liệu nghiên cứu)	21
4.1.1. Dataset thử nghiệm	21
4.1.2. Dataset huấn luyện chính	21
4.2. Lý thuyết về thuật toán XGBoost (Extreme Gradient Boosting)	22
4.2.1. Thuật toán Cây quyết định (Decision Tree)	22
4.2.2. Thuật toán XGBoost (Extreme Gradient Boosting)	23
4.2.3. Lý do lựa chọn XGBoost	23
4.3. Lý thuyết về kiến trúc Transformer	24
4.3.1. Giới thiệu về Transformer	24
4.3.2. Cơ chế Self-Attention	25
4.3.3. Multi-Head Attention	26
4.3.4. Biểu diễn dữ liệu đầu vào dạng bảng bằng Embedding	27
4.3.5. Lý do lựa chọn Transformer	28
4.4. Tiền xử lý dữ liệu	29
4.4.1. Tổng quan về tập dữ liệu	29
4.4.2. Encoding & Feature Extraction	31
4.4.3. EDA - Exploratory Data Analysis	32
4.4.4. Chuẩn hóa và biến đổi dữ liệu	32
4.4.5. Luồng xử lý tổng thể	33
4.5. Huấn luyện mô hình XGBoost	34
4.6. Huấn luyện mô hình Transformer	35
4.6.1. Token hóa dữ liệu bảng bằng những đơn điệu	35

4.6.2.	Transformer Encoder	37
4.6.3.	Làm phẳng (Flatten)	39
4.6.4.	Mạng nơ-ron phân loại nhị phân (Binary Classifier)	40
4.6.5.	Kiến trúc tổng thể mô hình	40
4.6.6.	Huấn luyện mô hình	41
Chương 5.	ĐÁNH GIÁ MÔ HÌNH	44
5.1.	Kết quả thực nghiệm và đánh giá	44
5.1.1.	Chia tập Train/Test	44
5.1.2.	Đánh giá mô hình trên dataset thử nghiệm	44
5.1.3.	Đánh giá mô hình trên Dataset huấn luyện chính	47
5.2.	Kết luận	50
5.3.	Hạn chế	51
Chương 6.	ĐỀ CƯƠNG KHÓA LUẬN	52
6.1.	Phát hiện Co-location Attack trong môi trường Serverless	52
6.2.	Phát hiện bất thường trong môi trường Serverless bằng Dynamic Knowledge Graph và Dynamic GNN	54

TÓM TẮT ĐỒ ÁN

Sự dịch chuyển từ các kiến trúc ứng dụng truyền thống sang mô hình điện toán không máy chủ (Serverless Computing) mang lại nhiều lợi ích về khả năng tự động mở rộng và tối ưu chi phí theo mô hình Pay-as-you-go. Tuy nhiên, đặc tính này cũng tạo điều kiện cho tấn công Denial of Wallet (DoW), trong đó kẻ tấn công liên tục gửi các yêu cầu hợp lệ nhằm khai thác cơ chế tự động mở rộng, làm gia tăng mức tiêu thụ tài nguyên và chi phí vận hành của hệ thống đám mây.

Nhằm giải quyết thách thức bảo mật đặc trưng này, đồ án tập trung nghiên cứu và xây dựng giải pháp ứng dụng trí tuệ nhân tạo để phát hiện hành vi tấn công Denial of Wallet trên dữ liệu lưu lượng mạng và dữ liệu thu thập từ hệ thống (telemetry). Nghiên cứu triển khai đánh giá và đối sánh hiệu năng giữa hai hướng tiếp cận: Mô hình học máy truyền thống XGBoost và mô hình học sâu Transformer được tùy biến riêng cho cấu trúc dữ liệu dạng bảng.

Kết quả thực nghiệm trên tập kiểm thử quy mô lớn cho thấy cả hai mô hình đều đạt hiệu năng xuất sắc với độ chính xác tổng thể xấp xỉ 99%. Trong đó, mô hình Transformer thể hiện ưu thế vượt trội về khả năng giảm thiểu cảnh báo sai với tỷ lệ False Positive thấp hơn đáng kể, giúp hạn chế tối đa việc chặn nhầm người dùng hợp lệ. Ngược lại, mô hình XGBoost đạt hiệu quả nhận diện Bot tấn công cao hơn và sở hữu tốc độ xử lý suy luận cực nhanh (nhanh hơn Transformer khoảng 7 lần), cực kỳ phù hợp cho các kịch bản phản ứng theo thời gian thực.

Trên cơ sở kết quả đạt được, đồ án đề xuất một kiến trúc hệ thống giám sát và phản ứng bất đồng bộ gần thời gian thực tích hợp với Kong API Gateway và OpenFaaS trên nền tảng Kubernetes. Cuối cùng, đồ án mở ra hai định hướng phát triển nghiên cứu tiềm năng cho Khóa luận tốt nghiệp bao gồm: Phát hiện tấn công đồng vị trí (Co-location Attack) và Phát hiện bất thường qua log hệ thống Serverless sử dụng Đồ thị tri thức động (Dynamic Knowledge Graph) kết hợp mạng Dynamic GNN.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Giới thiệu về Serverless Computing

Serverless Computing là mô hình điện toán đám mây cho phép nhà phát triển chỉ tập trung xây dựng logic ứng dụng, trong khi việc quản lý hạ tầng như cấp phát tài nguyên, mở rộng, cân bằng tải và bảo trì được nhà cung cấp dịch vụ tự động đảm nhiệm. Mặc dù vẫn sử dụng máy chủ (không hẳn là “Serverless” - không máy chủ), người dùng dịch vụ không cần trực tiếp quản lý chúng. Tài nguyên được cấp phát tự động theo nhu cầu thực tế, giúp tối ưu chi phí và khả năng mở rộng. Hai thành phần chính của Serverless là Function-as-a-Service (FaaS) và Backend-as-a-Service (BaaS).

1.1.1. Function-as-a-Service

Function-as-a-Service (FaaS) là mô hình cho phép triển khai các đơn vị mã nguồn nhỏ, độc lập (gọi là hàm - function), mỗi function đảm nhiệm một chức năng cụ thể như xử lý API, xác thực, gửi email hoặc xử lý dữ liệu. Function chỉ được thực thi khi có **sự kiện** kích hoạt, chẳng hạn như HTTP request, sự kiện từ API Gateway, thay đổi trong cơ sở dữ liệu, tệp được tải lên, thông điệp từ hàng đợi hoặc sự kiện theo lịch. Vì thế, kiến trúc Serverless được gọi là kiến trúc hướng sự kiện (event-driven).

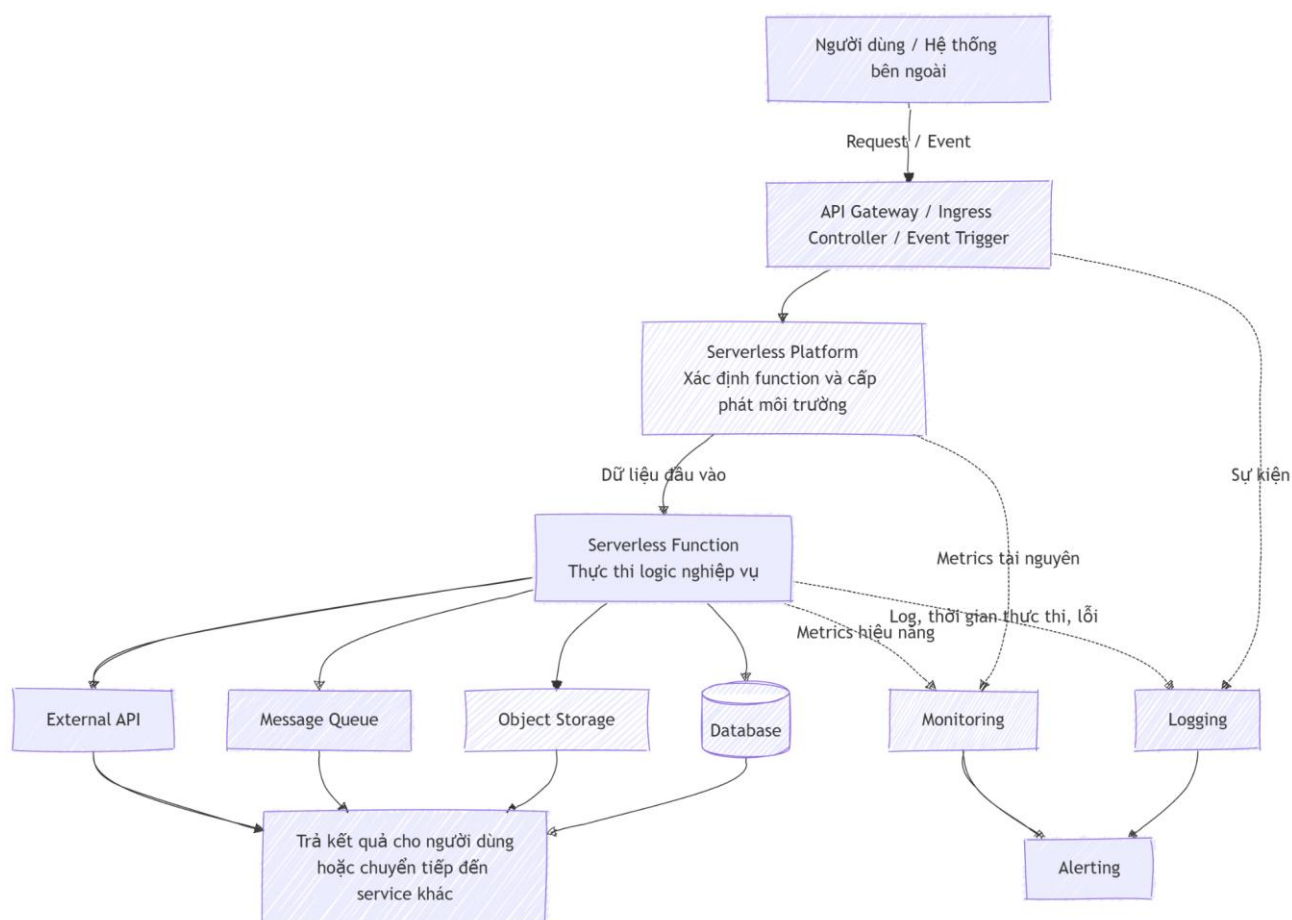
1.1.2. Backend-as-a-Service

Backend-as-a-Service (BaaS) là mô hình cung cấp sẵn các dịch vụ backend được quản lý bởi nhà cung cấp đám mây, cho phép nhà phát triển sử dụng trực tiếp thông qua API hoặc SDK mà không cần tự triển khai và vận hành. Các dịch vụ phổ biến gồm cơ sở dữ liệu, lưu trữ, xác thực người dùng, hàng đợi thông điệp, dịch vụ thông báo, ghi log và giám sát.

Có thể hiểu FaaS và BaaS là hai thành phần bổ trợ lẫn nhau. FaaS chịu trách nhiệm xử lý logic nghiệp vụ theo sự kiện, trong khi BaaS cung cấp các chức năng backend như lưu trữ, xác thực, hàng đợi và giám sát.

1.1.3. Luồng hoạt động cơ bản của Serverless

Hình dưới minh họa luồng xử lý của kiến trúc Serverless. Yêu cầu từ người dùng được chuyển đến cơ chế kích hoạt (event trigger), sau đó function được thực thi để xử lý nghiệp vụ và tương tác với các dịch vụ backend. Kết quả được trả về hoặc chuyển tiếp đi, đồng thời thông tin hoạt động được thu thập lại nhằm phục vụ giám sát.



Hình 1: Luồng xử lý cơ bản trong kiến trúc Serverless Computing

1.1.4. Các đặc tính cốt lõi của Serverless Computing

- Tự động mở rộng theo nhu cầu: Hệ thống tự động tăng/giảm số lượng instance hoặc function theo lưu lượng request thực tế để tối ưu tài

nguyên. Tuy nhiên, khả năng này vẫn bị giới hạn bởi hạn mức dịch vụ, cấu hình hệ thống, năng lực của cơ sở dữ liệu và băng thông mạng.

- Tính phi trạng thái (Stateless): Function không lưu trữ dữ liệu nghiệp vụ lâu dài trong bộ nhớ cục bộ. Mọi dữ liệu cần duy trì giữa các lần thực thi bắt buộc phải lưu ở hệ thống bên ngoài như cơ sở dữ liệu, cache, object storage hoặc message queue.
- Thanh toán theo mức độ sử dụng (Pay-as-you-go): Chi phí được tính thực tế dựa trên số lượt gọi, thời gian chạy, bộ nhớ cấp phát, băng thông hoặc dịch vụ phụ trợ. Mô hình này giúp tối ưu chi phí khi lưu lượng biến động, nhưng dễ gây rủi ro mất kiểm soát ngân sách nếu hệ thống bị lạm dụng (điển hình là kiểu tấn công Denial-of-Wallet)

1.1.5. Xu hướng chuyển dịch sang kiến trúc Serverless

Quá trình tiến hóa của kiến trúc ứng dụng trải qua ba giai đoạn: Từ mô hình Monolith truyền thống (đóng gói toàn bộ chức năng, chạy liên tục và phải nâng cấp hạ tầng thủ công), đến Microservices & Container (chia nhỏ thành các dịch vụ độc lập nhưng vẫn người dùng vẫn phải quản lý thủ công runtime, mô hình mạng và cụm Kubernetes) và hiện tại là Serverless (mức độ trừu tượng tối đa, giúp lập trình viên chỉ cần tập trung vào phát triển logic nghiệp vụ và chức năng ứng dụng, trong khi nhà cung cấp tự động quản lý, vận hành và mở rộng hạ tầng theo nhu cầu).

Tiêu chí	Monolith	Microservices/ Container	Serverless/FaaS
Đơn vị triển khai	Toàn bộ ứng dụng	Dịch vụ độc lập	Function nhỏ theo sự kiện
Quản lý máy chủ	Tổ chức tự quản lý	Tổ chức quản lý một phần lớn	Nền tảng quản lý phần lớn
Khả năng mở rộng	Thường thủ công hoặc theo VM	Có thể tự động qua orchestration	Tự động theo request hoặc event
Mô hình chi	Trả theo số máy chủ	Trả theo số node	Trả theo lượt gọi và

phí	hoặc VM chạy liên tục	hoặc container	tài nguyên sử dụng
Tốc độ triển khai	Chậm hơn do phạm vi thay đổi lớn	Nhanh hơn Monolith	Nhanh với thay đổi nhỏ, độc lập
Trọng tâm về bảo mật	Máy chủ, hệ điều hành, mạng	Container, cluster, service mesh	Code, IAM, event trigger, cấu hình, chi phí
Khả năng quan sát	Tập trung trên máy chủ lâu dài	Phân tán giữa nhiều service	Khó hơn do function ngắn hạn và phân tán

Bảng 1: So sánh kiến trúc truyền thống, Microservices và Serverless

Sự dịch chuyển sang Serverless xuất phát từ nhu cầu tối ưu vận hành và rút ngắn thời gian phát triển ứng dụng. Tuy nhiên, Serverless không phù hợp với mọi loại ứng dụng do các hạn chế như cold start, giới hạn thời gian thực thi và sự phụ thuộc vào nền tảng cung cấp. Vì vậy, việc lựa chọn mô hình Serverless cần được cân nhắc dựa trên đặc điểm của từng hệ thống.

1.2. Mô hình chia sẻ trách nhiệm trong kiến trúc Serverless

Mô hình chia sẻ trách nhiệm (Shared Responsibility Model) phân định trách nhiệm bảo mật giữa nhà cung cấp dịch vụ đám mây và khách hàng. Trong môi trường Serverless được quản lý hoàn toàn (Managed Services), nhà cung cấp chịu trách nhiệm bảo vệ hạ tầng, bao gồm trung tâm dữ liệu, mạng, máy chủ, runtime và các dịch vụ nền tảng, trong khi khách hàng chịu trách nhiệm bảo mật ứng dụng, dữ liệu, cấu hình và quyền truy cập. Mức độ phân chia trách nhiệm phụ thuộc vào mô hình triển khai: Với các dịch vụ FaaS Managed Services như AWS Lambda, Azure Functions hay Google Cloud Functions, phần lớn hạ tầng do nhà cung cấp đảm nhiệm. Ngược lại, khi sử dụng các nền tảng tự triển khai như OpenFaaS kết hợp Kubernetes, tổ chức vẫn phải quản lý nhiều thành phần hạ tầng như cluster, container runtime, mạng và cơ chế mở rộng.

Việc sử dụng Serverless không loại bỏ trách nhiệm bảo mật của tổ chức mà chỉ chuyển trọng tâm từ bảo vệ máy chủ sang bảo vệ mã nguồn, danh tính, quyền truy cập, dữ liệu và cấu hình triển khai. Do đó, đội ngũ phát triển cần đảm bảo an toàn cho function, áp dụng nguyên tắc đặc quyền tối thiểu, quản lý secret, bảo vệ API và event trigger, đồng thời triển khai cơ chế giám sát và kiểm soát chi phí để duy trì tính an toàn và ổn định của hệ thống.

1.3. Thách thức bảo mật trong kiến trúc Serverless

Mặc dù Serverless mang lại nhiều lợi ích về khả năng mở rộng và tốc độ triển khai, mô hình này cũng đặt ra các thách thức bảo mật mới do kiến trúc phân tán và cơ chế kích hoạt hướng sự kiện. So với hệ thống truyền thống, số lượng function, endpoint và dịch vụ backend lớn hơn làm mở rộng bề mặt tấn công, đồng thời tăng nguy cơ phát sinh lỗi cấu hình (misconfiguration) và khó khăn trong việc theo dõi luồng xử lý.

Bên cạnh đó, đặc tính thực thi ngắn hạn của function khiến việc giám sát, thu thập bằng chứng và điều tra sự cố trở nên phức tạp hơn, đòi hỏi hệ thống phải có cơ chế quan sát (observability) hiệu quả. Đồng thời, các cơ chế phòng thủ truyền thống như Firewall hay IPS không còn đủ để bảo vệ toàn diện một hệ thống quá rộng và phức tạp như vậy. Vì thế, bảo mật Serverless cần kết hợp các biện pháp kiểm soát truy cập, giám sát và phát hiện bất thường nhằm bảo vệ hệ thống một cách toàn diện.

1.4. Cuộc tấn công Denial of Wallet

Denial of Wallet (DoW) là hình thức tấn công nhằm gây thiệt hại kinh tế cho các hệ thống sử dụng cơ chế Pay-as-you-go, và là kiểu tấn công đặc trưng nhắm vào kiến trúc Serverless. Khác với Denial of Service (DoS) vốn tập trung vào làm gián đoạn dịch vụ, DoW khai thác cơ chế tự động mở rộng (autoscaling) và mô hình thanh toán theo mức độ sử dụng (Pay-as-you-Go) để làm gia tăng chi phí vận hành của hệ thống.

Trong môi trường Serverless, mỗi yêu cầu hoặc sự kiện có thể kích hoạt function và sử dụng các dịch vụ phụ trợ như cơ sở dữ liệu, lưu trữ, hàng đợi hay API bên thứ ba. Khi kẻ tấn công liên tục tạo ra các request hoặc event hợp lệ, hệ thống vẫn có thể hoạt động bình thường nhờ autoscaling nhưng phải tiêu thụ nhiều tài nguyên hơn, dẫn đến chi phí vận hành tăng bất thường.

DoW thường khó phát hiện hơn các cuộc tấn công DoS truyền thống vì lưu lượng tấn công có thể hoàn toàn hợp lệ về mặt giao thức và không chứa mã độc. Điều này đòi hỏi các hệ thống Serverless phải kết hợp nhiều cơ chế bảo vệ như kiểm soát truy cập, giới hạn tốc độ, giám sát và phát hiện bất thường nhằm hạn chế nguy cơ lạm dụng tài nguyên và kiểm soát chi phí vận hành.

1.5. Mục tiêu

Xuất phát từ những thách thức bảo mật và nguy cơ thiệt hại kinh tế do tấn công Denial of Wallet trong môi trường Serverless, đề án tập trung nghiên cứu khả năng ứng dụng các kỹ thuật học máy và học sâu nhằm phát hiện các hành vi tấn công trên lưu lượng mạng. Cụ thể, đề tài hướng đến các mục tiêu sau:

- Xây dựng và huấn luyện hai mô hình phát hiện tấn công Denial of Wallet gồm mô hình học máy XGBoost và mô hình học sâu Transformer tùy biến sử dụng Tokenizer kết hợp Monotonic Embedding (nhúng đơn điệu).
- Đánh giá và so sánh hiệu năng của hai mô hình thông qua các chỉ số như Accuracy, Precision, Recall, F1-score và thời gian suy luận nhằm xác định mô hình phù hợp cho bài toán phát hiện Denial of Wallet.
- Đề xuất định hướng áp dụng mô hình tối ưu vào hệ thống giám sát Serverless, góp phần cân bằng giữa khả năng phát hiện tấn công, giảm cảnh báo sai và đáp ứng yêu cầu xử lý theo thời gian thực.

Chương 2. CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Tổng quan về tấn công Denial of Wallet trong môi trường Serverless

Trong nghiên cứu *Denial of Wallet in Serverless Computing: A Systematic Literature Review* (2025), các tác giả đã thực hiện hệ thống hóa các công trình nghiên cứu về tấn công Denial of Wallet (DoW) trong môi trường Serverless bằng phương pháp PRISMA. Nhóm sử dụng nghiên cứu này làm tài liệu tham khảo tổng quan nhằm nắm bắt đặc điểm của tấn công DoW, các hướng tiếp cận phát hiện và những khoảng trống nghiên cứu hiện nay, từ đó định hướng xây dựng và đánh giá mô hình phát hiện trong phạm vi của đề án.

2.2. Phân loại tấn công DoW trong môi trường Serverless

Trong nghiên cứu *Distributed Denial of Wallet Attack on Serverless Pay-as-you-go Model* (2022), các tác giả đã chia Denial of Wallet thành ba dạng tấn công gồm **Blast DDoW**, **Continual Inconspicuous DDoW** và **Background Chained DDoW**.

Blast DDoW là hình thức gửi một lượng lớn yêu cầu trong thời gian ngắn nhằm kích hoạt tối đa cơ chế Auto-scaling, từ đó làm tăng nhanh chi phí vận hành và có thể gây ảnh hưởng đến tính sẵn sàng của dịch vụ. Continual Inconspicuous DDoW sử dụng lưu lượng thấp nhưng duy trì trong thời gian dài, khiến cuộc tấn công khó bị phát hiện trong khi vẫn tạo ra chi phí đáng kể. Trong khi đó, Background Chained DDoW khai thác kiến trúc hướng sự kiện của Serverless bằng cách kích hoạt gián tiếp chuỗi các function thông qua các thao tác hợp lệ từ người dùng, làm phát sinh nhiều lần thực thi ở phía sau mà không cần truy cập trực tiếp vào endpoint của các function.

Để đánh giá tác động của các hình thức tấn công này, nhóm tác giả triển khai thực nghiệm trên Google Cloud Functions. Kết quả cho thấy Blast DDoW có thể làm phát sinh chi phí khoảng 856,75 USD/ngày với trung bình 1.982 request/giây, trong khi Continual Inconspicuous DDoW chỉ tạo khoảng 155

request/giây nhưng vẫn gây thiệt hại khoảng 63,65 USD/ngày nếu duy trì liên tục. Điều này chứng minh rằng ngay cả các lưu lượng hợp lệ có cường độ thấp cũng có thể gây tổn thất tài chính đáng kể trong môi trường Serverless.

Đóng góp quan trọng nhất của nghiên cứu là xây dựng hệ thống phân loại các biến thể DoW và chứng minh bằng thực nghiệm khả năng gây thiệt hại kinh tế của từng loại tấn công. Trong đề tài này, nhóm sẽ tập trung vào phát hiện kiểu tấn công **Continual Inconspicuous**, tấn công sử dụng lưu lượng thấp nhưng duy trì liên tục trong thời gian dài.

2.3. DoWTS - Bộ sinh dữ liệu phục vụ nghiên cứu về DoW

Trong nghiên cứu *DoWTS – Denial-of-Wallet Test Simulator: Synthetic data generation for preemptive defence* (2023), giáo sư Kelly và cộng sự nhận thấy rằng mặc dù Denial of Wallet (DoW) đã được xác định là một mối đe dọa đặc trưng của kiến trúc Serverless, các nghiên cứu về phát hiện tấn công bằng trí tuệ nhân tạo vẫn gặp khó khăn do thiếu bộ dữ liệu công khai. Việc thu thập dữ liệu thực tế đòi hỏi triển khai hệ thống Serverless trong thời gian dài và phát sinh chi phí lớn, trong khi các cuộc tấn công DoW ngoài thực tế rất hiếm khi được công bố. Vì vậy, nhóm tác giả đề xuất công cụ **DoWTS** (Denial of Wallet Test Simulator) nhằm tạo ra dữ liệu mô phỏng có đặc tính gần với lưu lượng thực tế để phục vụ nghiên cứu và huấn luyện các mô hình phát hiện tấn công.

Khác với việc tạo dữ liệu ngẫu nhiên, DoWTS được thiết kế để mô phỏng hành vi truy cập của người dùng trong môi trường Serverless. Bộ sinh dữ liệu xây dựng lưu lượng nền (background traffic) dựa trên các tham số như số lượng người dùng, tần suất truy cập, chu kỳ ngày-đêm, biến động theo ngày trong tuần và mức độ ngẫu nhiên của hành vi người dùng. Trên nền lưu lượng này, các kịch bản DoW được chèn vào với các tham số có thể cấu hình như thời điểm bắt đầu, thời gian duy trì, số lượng bot và tốc độ gửi yêu cầu, từ đó tạo ra các tập dữ liệu chứa cả lưu lượng hợp lệ và lưu lượng tấn công. Cách

tiếp cận này cho phép sinh ra nhiều kịch bản với quy mô và cường độ khác nhau mà không cần triển khai hệ thống thực tế.

Để đánh giá tính chân thực của dữ liệu sinh ra, nhóm tác giả sử dụng hai bộ dữ liệu truy cập thực tế làm chuẩn so sánh. Các tập dữ liệu tổng hợp được kiểm định bằng nhiều phương pháp thống kê như Kolmogorov-Smirnov Test (KS Test), Wasserstein Distance và Jensen-Shannon Divergence, đồng thời kết hợp đánh giá trực quan thông qua phân bố lưu lượng theo thời gian. Kết quả cho thấy dữ liệu do DoWTS tạo ra có phân bố tương đồng với dữ liệu thực, thể hiện qua các giá trị Wasserstein Distance và Jensen-Shannon Divergence thấp, trong khi xu hướng lưu lượng theo chu kỳ ngày-đêm và biến động theo tuần cũng được tái hiện tương đối chính xác.

Một đóng góp quan trọng khác của nghiên cứu là xây dựng quy trình tạo dữ liệu có khả năng mở rộng và tái sử dụng. Thay vì chỉ tạo ra một bộ dữ liệu cố định, DoWTS cho phép điều chỉnh các tham số mô phỏng để sinh ra nhiều tập dữ liệu phục vụ các mục đích nghiên cứu khác nhau. Điều này giúp khắc phục hạn chế về số lượng mẫu trong lĩnh vực phát hiện DoW, đồng thời tạo nền tảng cho việc huấn luyện và đánh giá các mô hình Machine Learning và Deep Learning trong các nghiên cứu tiếp theo.

2.4. Xây dựng bộ dữ liệu chuyên biệt về DoW

Trong nghiên cứu *Generation of a dataset for DoW attack detection in serverless architectures* (2024), các tác giả nhận thấy rằng mặc dù Denial of Wallet (DoW) ngày càng được xem là một trong những mối đe dọa đặc trưng của kiến trúc Serverless, việc nghiên cứu các mô hình phát hiện bằng Machine Learning và Deep Learning vẫn gặp nhiều khó khăn do thiếu bộ dữ liệu công khai. Chính khoảng trống này đã thúc đẩy nhóm tác giả xây dựng một bộ dữ liệu tổng hợp chuyên biệt dành cho bài toán phát hiện DoW.

Để giải quyết vấn đề trên, nghiên cứu sử dụng *DoWTS (Denial of Wallet Test Simulator)*, công cụ tới từ bài báo phía trên nhằm mô phỏng hoạt động của một hệ thống Serverless và sinh ra lưu lượng truy cập gần với thực tế. Công cụ cho phép cấu hình nhiều tham số như số lượng người dùng, bot, thời điểm bắt đầu tấn công, tốc độ gửi yêu cầu và thời gian duy trì cuộc tấn công, giúp sinh ra đồng thời cả lưu lượng hợp lệ và lưu lượng tấn công trong cùng một môi trường mô phỏng.

Quá trình xây dựng bộ dữ liệu được thực hiện qua ba giai đoạn.

- Giai đoạn đầu tiên được sinh ra từ DoWTS. Thành phần này cung cấp các thông tin cơ bản như địa chỉ IP, nhãn phân loại bot hoặc người dùng hợp lệ, thời gian gọi hàm, loại sự kiện kích hoạt hàm (*functionTrigger*) và mã định danh hàm (*FunctionId*). Nhóm tác giả sử dụng biến *FunctionId* của nguồn này làm **khóa chính** để kết hợp dữ liệu từ nhiều nguồn khác nhau.
- Nhóm tác giả bổ sung thêm các đặc trưng phản ánh hiệu năng của hệ thống Serverless từ bộ dữ liệu "*Concurrent Execution Monte Carlo Serverless Functions Across AWS, Google, IBM and Alibaba*" trên IEEE Dataport. Dữ liệu này cung cấp các chỉ số liên quan đến độ trễ và hiệu năng thực thi của hệ thống serverless, bao gồm *SubmitTime*, *RTT*, *InvocationDelay*, *ResponseDelay*, *FunctionDuration*, *ActiveFunctionsAtRequest* và *ActiveFunctionsAtResponse*.
- Cuối cùng, bộ dữ liệu được mở rộng bằng các đặc trưng liên quan đến mức sử dụng tài nguyên của máy ảo từ *Azure Public Dataset* do Microsoft Azure công bố. Nguồn dữ liệu này bổ sung các thông tin về tài nguyên phần cứng và telemetry của hệ thống như mức sử dụng CPU tối đa (*maxcpu*), CPU tối đa ở ngưỡng phân vị 95% (*p95maxcpu*), CPU trung bình (*avgcpu*), phân khúc bộ nhớ máy ảo (*vmmemorybucket*) và phân khúc số lượng lõi xử lý của máy ảo (*vmcorecountbucket*).

Đóng góp quan trọng nhất của nghiên cứu là cung cấp một bộ dữ liệu mở đầu tiên được thiết kế chuyên biệt cho bài toán phát hiện Denial of Wallet trong môi trường Serverless. Bộ dữ liệu không chỉ hỗ trợ đánh giá các mô hình Machine Learning và Deep Learning mà còn tạo nền tảng cho các nghiên cứu về tối ưu tài nguyên, dự đoán chi phí và phát hiện bất thường trong kiến trúc Serverless. Đây là một trong hai tập dữ liệu được nhóm sử dụng trong đồ án.

2.5. Mở rộng bộ dữ liệu về DoW bằng Generative AI và GAN

Tiếp nối nghiên cứu về bộ dữ liệu DoW trong Serverless, các tác giả của bài báo phía trên nhận thấy rằng mặc dù bộ dữ liệu được xây dựng ở nghiên cứu trước đã tạo nền tảng cho việc phát triển các mô hình phát hiện Denial of Wallet, quy mô dữ liệu vẫn còn tương đối hạn chế đối với các mô hình Machine Learning và đặc biệt là Deep Learning. Các mô hình học sâu thường yêu cầu lượng dữ liệu lớn và đa dạng để giảm hiện tượng overfitting và nâng cao khả năng tổng quát hóa. Do đó, nhóm tác giả đề xuất sử dụng Generative Artificial Intelligence (Generative AI) trong bài báo *Data augmentation with Generative AI for DoW attack detection in serverless architectures* (2024) để mở rộng bộ dữ liệu hiện có thay vì tiếp tục thu thập hoặc mô phỏng dữ liệu mới từ đầu.

Nghiên cứu lựa chọn Generative Adversarial Network (GAN) làm nền tảng sinh dữ liệu tổng hợp. Mô hình gồm hai mạng nơ-ron là Generator và Discriminator hoạt động đối kháng với nhau. Trong quá trình huấn luyện, Generator liên tục tạo ra các mẫu dữ liệu mới có phân bố gần với dữ liệu gốc, trong khi Discriminator học cách phân biệt dữ liệu thật và dữ liệu sinh ra. Quá trình cạnh tranh này giúp các mẫu dữ liệu tổng hợp ngày càng giống với dữ liệu thực, từ đó có thể được sử dụng để tăng kích thước tập huấn luyện mà vẫn duy trì được các đặc trưng thống kê của bộ dữ liệu ban đầu.

Khác với việc tạo ra một cấu trúc dữ liệu mới, nghiên cứu vẫn kế thừa toàn bộ đặc trưng của bộ dữ liệu trước nhằm đảm bảo khả năng tương thích với các

mô hình phát hiện DoW. Điểm cải tiến lớn nhất nằm ở quy mô dữ liệu, khi bộ dữ liệu trước chỉ gồm khoảng 187.087 bản ghi, bộ dữ liệu mới được mở rộng lên 2.000.000 giao dịch mỗi bộ dữ liệu, với tổng cộng 7 bộ dữ liệu như thế, có tỉ lệ Bot lần lượt là 5%, 10%, 15%, 20%, 25%, 30% và 35%. Việc này giúp bộ dữ liệu phản ánh sát hơn đặc điểm của môi trường thực tế, nơi các cuộc tấn công thường chỉ chiếm một tỷ lệ nhỏ trong tổng lưu lượng hệ thống, khi các nhà nghiên cứu có thể dễ dàng chọn tỉ lệ Bot phù hợp.

Ngoài việc tăng số lượng mẫu, nghiên cứu cũng phân tích các đặc trưng thống kê của dữ liệu sau khi mở rộng. Kết quả cho thấy các đặc trưng về thời gian thực thi, phân bố Function, tải hệ thống và tương quan giữa các thuộc tính vẫn được bảo toàn sau quá trình sinh dữ liệu. Điều này cho thấy Generative AI không chỉ giúp tăng quy mô tập dữ liệu mà còn duy trì được phân bố của dữ liệu gốc, tạo điều kiện thuận lợi cho việc huấn luyện các mô hình Machine Learning và Deep Learning có độ phức tạp cao. Đây cũng là một trong hai tập dữ liệu được nhóm sử dụng trong đồ án.

2.6. Đóng góp của nhóm

Dựa trên các nghiên cứu đã khảo sát, nhóm thực hiện đề tài với ba đóng góp chính sau:

- **Xây dựng và đánh giá** hai hướng tiếp cận phát hiện Denial of Wallet gồm mô hình học máy truyền thống XGBoost (Shallow Learning) và mô hình học sâu Transformer được tùy biến cho dữ liệu dạng bảng. Việc đối sánh hai mô hình giúp đánh giá khả năng ứng dụng của từng hướng tiếp cận đối với bài toán phát hiện DoW trong môi trường Serverless.
- **Lựa chọn hướng tiếp cận dữ liệu khác** với phần lớn các nghiên cứu hiện nay. Trong khi đa số các công trình phát hiện Denial of Wallet bằng Machine Learning và Deep Learning đều sử dụng bộ dữ liệu từ nghiên cứu *Generation of a dataset for DoW attack detection in serverless architectures*, nhóm lựa chọn sử dụng bộ dữ liệu mở rộng trong *Data*

augmentation with Generative AI for DoW attack detection in serverless architectures. Quyết định này xuất phát từ việc bộ dữ liệu gốc vẫn còn rất nhiều hạn chế (sẽ được phân tích ở phần sau). Nhóm kỳ vọng việc sử dụng bộ dữ liệu mở rộng sẽ giúp nâng cao độ tin cậy của quá trình huấn luyện, đánh giá mô hình và tăng giá trị tham khảo của kết quả nghiên cứu.

- **Đề xuất định hướng triển khai** mô hình trong môi trường Serverless thực tế, bao gồm vị trí tích hợp trong kiến trúc giám sát, quy trình xử lý lưu lượng và cơ chế phát hiện theo thời gian thực. Qua đó, kết quả nghiên cứu không chỉ dừng ở việc đánh giá mô hình mà còn hướng tới khả năng ứng dụng trong các hệ thống Serverless nhằm hỗ trợ phát hiện sớm các hành vi Denial of Wallet.

Chương 3. KIẾN TRÚC ĐỀ XUẤT

3.1. Mục tiêu thiết kế kiến trúc

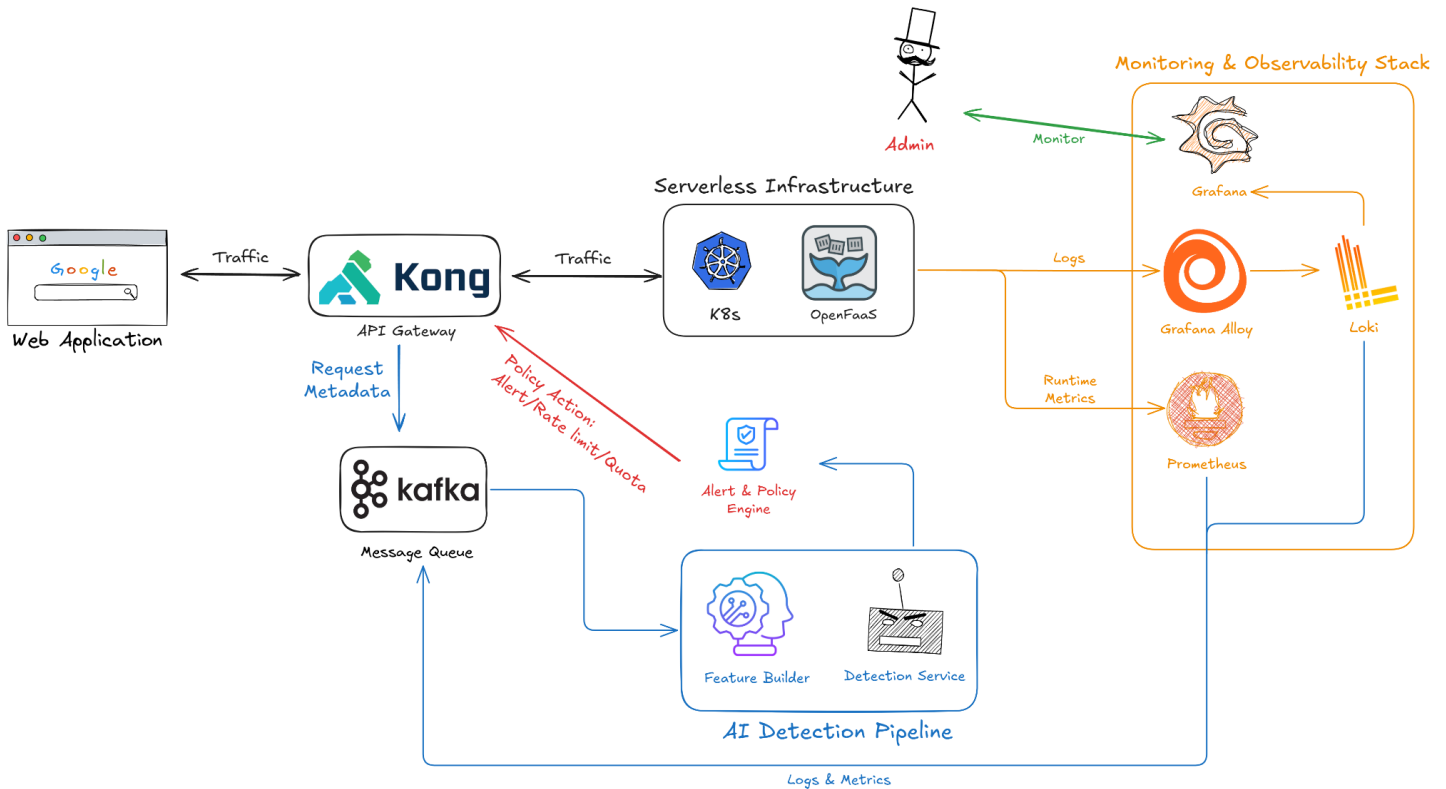
Denial of Wallet không chỉ thể hiện qua số lượng request tăng bất thường mà còn có thể liên quan đến thời gian thực thi function, mức sử dụng tài nguyên, khả năng autoscaling và hoạt động của các dịch vụ phụ trợ. Vì vậy, việc phát hiện cần kết hợp nhiều nguồn telemetry thay vì chỉ dựa trên lưu lượng mạng hoặc địa chỉ IP.

Vì vậy, các nguyên tắc thiết kế chính gồm:

- Không làm ảnh hưởng đáng kể đến luồng xử lý request chính của ứng dụng.
- Thu thập đồng thời request metadata, log và runtime metrics.
- Xử lý dữ liệu phát hiện theo cơ chế bất đồng bộ.
- Tổng hợp hành vi theo cửa sổ thời gian để nhận biết các cuộc tấn công kéo dài.
- Tách biệt giữa hoạt động phát hiện và hoạt động phản ứng nhằm hạn chế chặn nhầm người dùng hợp lệ.

Kiến trúc trong chương này được trình bày dưới dạng đề xuất, đóng vai trò định hướng cho việc tích hợp cơ chế phát hiện Denial of Wallet bằng trí tuệ nhân tạo vào môi trường Serverless thực tế. Kiến trúc được xây dựng nhằm xác định vị trí thu thập dữ liệu, luồng chuyển dữ liệu, cách tổng hợp đặc trưng và cơ chế phản ứng khi phát hiện hành vi có nguy cơ gây gia tăng chi phí vận hành.

3.2. Kiến trúc tổng quan



Hình 2: Kiến trúc hệ thống phát hiện Denial of Wallet trong môi trường Serverless

Kiến trúc được thiết kế theo hướng tách biệt ba luồng: luồng xử lý nghiệp vụ, luồng thu thập telemetry và luồng phát hiện - phản ứng. Cách tổ chức này giúp việc phân tích không làm tăng đáng kể độ trễ phản hồi của ứng dụng:

- Kong API Gateway: Tiếp nhận request, định tuyến và thu thập request metadata.
- Kubernetes và OpenFaaS: Thực thi Serverless Function và quản lý khả năng mở rộng.
- Grafana Alloy, Loki, Prometheus, Grafana: Thu thập log, metric, trực quan hóa và hỗ trợ giám sát.
- Kafka: Tách luồng phân tích khỏi luồng request chính.
- Detection Pipeline: Tổng hợp đặc trưng và đánh giá nguy cơ Denial of Wallet.
- Alert & Policy Engine: Sinh cảnh báo và điều phối chính sách phản ứng.

3.3. Luồng hoạt động của hệ thống

Luồng hoạt động của hệ thống đề xuất được mô tả qua các bước sau:

- Bước 1: Tiếp nhận request

Người dùng gửi request đến Kong API Gateway. Kong xác thực, áp dụng các policy cơ bản và định tuyến request đến function tương ứng trên OpenFaaS.

- Bước 2: Thu thập request metadata & telemetry runtime

Song song với luồng xử lý chính, Kong ghi nhận metadata request như thời điểm, endpoint, response code, latency và nguồn truy cập. Dữ liệu này được gửi đến topic request-events của Kafka.

Prometheus thu thập metric từ OpenFaaS, Kubernetes và các thành phần liên quan. Metric sau đó được chuyển đến topic runtime-metrics.

- Bước 3: Xây dựng đặc trưng

Feature Builder kết hợp request event với telemetry runtime theo các cửa sổ thời gian. Kết quả là các vector đặc trưng phản ánh hành vi truy cập, tình trạng thực thi function và mức sử dụng tài nguyên.

- Bước 4: Suy luận bằng mô hình AI

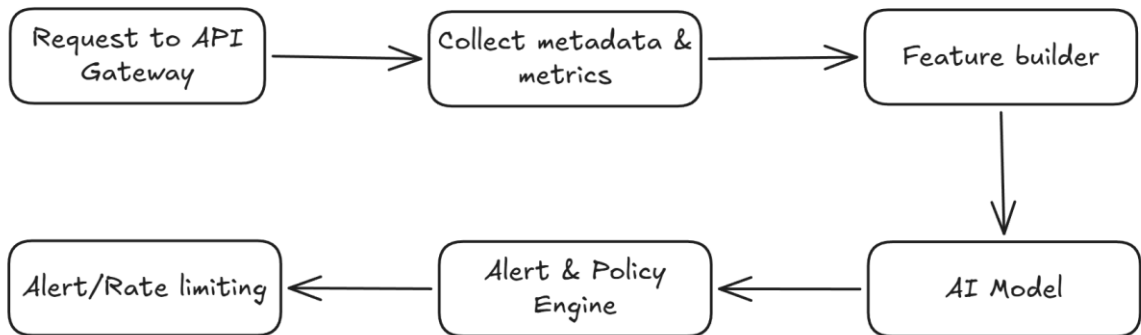
Detection Service sử dụng mô hình đã được huấn luyện để đánh giá vector đặc trưng. Hệ thống sinh ra nhãn dự đoán, điểm rủi ro và thông tin liên quan đến mức độ ảnh hưởng tài nguyên.

- Bước 5: Đánh giá chính sách

Alert & Policy Engine kiểm tra điểm rủi ro, endpoint bị ảnh hưởng và chính sách hiện hành. Nếu nguy cơ thấp, hệ thống chỉ ghi log và theo dõi. Nếu nguy cơ cao, hệ thống có thể gửi yêu cầu áp dụng rate limiting hoặc giới hạn tạm thời đến Kong.

- Bước 6: Giám sát và điều tra

Quản trị viên theo dõi dashboard Grafana, kiểm tra log trên Loki và metric trên Prometheus để xác nhận cảnh báo, đánh giá tác động và điều chỉnh chính sách khi cần thiết.



Hình 3: Luồng phát hiện và phản ứng đối với hành vi Denial of Wallet

3.4. Ảnh xạ đặc trưng vào môi trường triển khai

Bộ dữ liệu sử dụng trong đồ án bao gồm các nhóm đặc trưng về traffic, thực thi function và mức sử dụng tài nguyên. Trong kiến trúc đề xuất, các nhóm đặc trưng này có thể được thu thập tương ứng từ các nguồn sau:

Nhóm đặc trưng	Ví dụ đặc trưng	Nguồn dữ liệu đề xuất
Traffic	IP, timestamp, FunctionId, trigger	Kong access log, route metadata, request event
Request latency	RTT, ResponseDelay, response code	Kong log, tracing hoặc gateway metric
Function execution	FunctionDuration, InvocationDelay	OpenFaaS metric, application metric
Function concurrency	ActiveFunctionsAtRequest/Response	OpenFaaS, Kubernetes, Prometheus
Resource telemetry	maxcpu, avgcpu, p95maxcpu	Prometheus, node exporter, container metric
Hạ tầng	vmcategory, vmcorecountbucket,	Kubernetes node metadata, cloud instance metadata

	vmmemorybucket	
Rolling feature	Số request/IP trong 1 phút và 1 giờ	Feature Builder kết hợp Window State

Bảng 2: Bảng ánh xạ đặc trưng trong Dataset vào nguồn dữ liệu thực tế

3.5. Các vấn đề khi vận hành thực tế

Kiến trúc hệ thống được đề xuất nhằm chứng minh khả năng kết hợp mô hình AI với hệ thống giám sát Serverless. Tuy nhiên, khi triển khai trong môi trường thực tế, cần xem xét các giới hạn sau:

- Bộ dữ liệu huấn luyện là dữ liệu mô phỏng, do đó có thể chưa phản ánh đầy đủ hành vi của người dùng thật và các cuộc tấn công thực tế.
- Một số đặc trưng trong dataset có nguy cơ mang thông tin thời gian hoặc tần suất đặc thù của môi trường mô phỏng, dẫn đến nguy cơ giảm khả năng tổng quát hóa.
- Pipeline phát hiện tạo thêm chi phí cho việc thu thập, lưu trữ và xử lý telemetry.
- Hệ thống cần bảo vệ dữ liệu nhạy cảm như IP, token hoặc định danh người dùng bằng phương pháp băm, ẩn danh hoặc giảm thiểu dữ liệu thu thập.
- Các quyết định phản ứng tự động cần được áp dụng thận trọng để tránh gây ảnh hưởng đến người dùng hợp lệ.
- Detection Service cũng cần được giám sát và mở rộng độc lập để tránh trở thành điểm nghẽn khi lưu lượng tăng cao.

Trong giai đoạn đầu, hệ thống nên hoạt động ở chế độ cảnh báo và quan sát. Sau khi có đủ dữ liệu thực tế để đánh giá false positive, false negative và độ trễ suy luận, hệ thống có thể dần mở rộng sang cơ chế phản ứng bán tự động hoặc tự động.

Một số cột trong dataset có thể không tồn tại hoàn toàn giống trong môi trường triển khai thực tế. Do đó, quá trình triển khai cần xây dựng phép ánh xạ giữa feature gốc và telemetry tương đương. Ví dụ, *FunctionId* có thể được thay bằng *function name* hoặc *route identifier*, *vmcategory* có thể được thay bằng *node pool* hoặc *instance type* của cụm Kubernetes.

Việc bảo đảm sự nhất quán giữa feature dùng trong huấn luyện và feature dùng khi suy luận là yêu cầu quan trọng. Mọi phép encoding, chuẩn hóa và biến đổi dữ liệu ở runtime cần tuân theo đúng pipeline đã sử dụng trong giai đoạn huấn luyện.

Tóm lại, kiến trúc đề xuất giúp chuyển kết quả nghiên cứu từ môi trường huấn luyện offline sang một mô hình giám sát Serverless gần thời gian thực. Kiến trúc này cũng tạo nền tảng để tiếp tục phát triển các hướng như phát hiện event storm, API abuse, retry loop, bất thường autoscaling và rủi ro chi phí trong môi trường cloud-native.

Chương 4. HUẤN LUYỆN MÔ HÌNH

4.1. Tổng quan về Dataset (Dữ liệu nghiên cứu)

4.1.1. Dataset thử nghiệm

Đây là tập dữ liệu từ bài báo: *Generation of a dataset for DoW attack detection in serverless architectures* của tác giả Ortega Candel và cộng sự, nhóm đã nhắc tới ở [phần 2.4](#).

- Bộ dữ liệu có 187.087 bản ghi. Dù chứa đến 187.087 bản ghi, dữ liệu này có sự mất cân bằng lớp cực kỳ phi thực tế, khi mẫu tấn công chiếm 70,06% (131.072 mẫu), trong khi mẫu hợp lệ chỉ chiếm 29,94% (56.015 mẫu).
- Mức độ dư thừa dữ liệu cũng rất cao do lượng giá trị phân biệt (unique values) rất thấp với nhiều cột.
- Đặc biệt, hiện tượng rò rỉ dữ liệu (Data Leakage) xuất hiện rõ rệt khi mô hình phụ thuộc quá mức vào các đặc trưng thời gian và tần suất IP để dự đoán.

Vì những lý do trên, nhóm quyết định chỉ sử dụng bộ dữ liệu này cho mục đích kiểm chứng tính khả thi và so sánh hiệu năng thô giữa các thuật toán.

4.1.2. Dataset huấn luyện chính

Đây là tập dữ liệu từ bài báo: *Data augmentation with Generative AI for DoW attack detection in serverless architectures* của tác giả Ortega Candel và cộng sự, nhóm đã nhắc tới ở [phần 2.5](#).

- Bộ dữ liệu này có các trường **giống hệt** với bộ dữ liệu phía trên, nhưng mức độ đa dạng và chất lượng của bộ dữ liệu được cải thiện đáng kể. Nhiều thuộc tính có số lượng giá trị phân biệt (unique values) cao hơn rất nhiều, hiện tượng lặp lại bản ghi giảm mạnh so với bộ dữ liệu gốc và các đặc trưng được phân bố tự nhiên hơn.

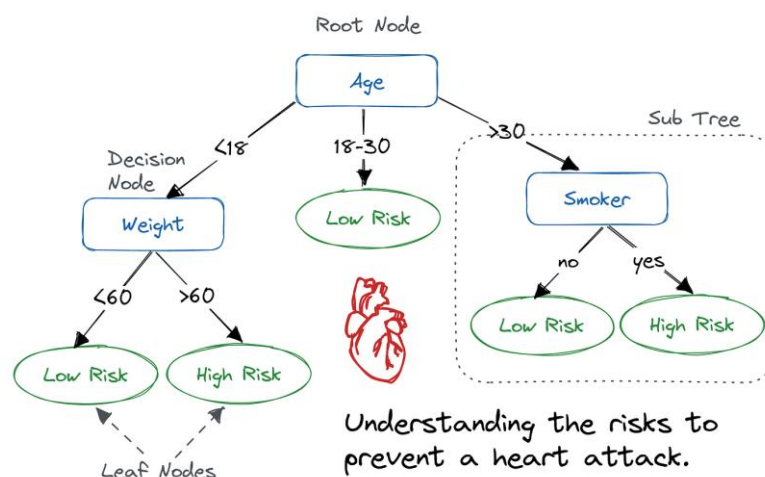
- Tuy nhiên, dataset này vẫn tồn tại hạn chế, khi phần lớn lưu lượng hợp lệ xuất hiện trong khoảng 00:00–03:00, trong khi lưu lượng bot tập trung chủ yếu từ 03:00–17:00.
- Nhóm tác giả đã công bố 7 bộ dữ liệu với quy mô 2.000.000 mẫu cho mỗi bộ, tương ứng với các tỷ lệ lưu lượng tấn công (bot traffic) lần lượt là 5%, 10%, 15%, 20%, 25%, 30% và 35%.

Nhóm sử dụng bộ dữ liệu này làm nguồn dữ liệu chính cho quá trình huấn luyện, đánh giá và so sánh các mô hình học máy.

4.2. Lý thuyết về thuật toán XGBoost (Extreme Gradient Boosting)

4.2.1. Thuật toán Cây quyết định (Decision Tree)

- Cây quyết định (Decision Tree) là một thuật toán học máy có giám sát phổ biến cho các bài toán phân loại và hồi quy. Thuật toán mô phỏng lại quá trình ra quyết định của con người dưới dạng một cấu trúc cây, trong đó mỗi nút biểu diễn một điều kiện kiểm tra trên thuộc tính dữ liệu, mỗi nhánh thể hiện kết quả của điều kiện đó và mỗi nút lá biểu diễn kết quả dự đoán cuối cùng. Nhờ tính đơn giản, hiệu suất và tốc độ xử lý cao cũng sự chính xác, cây quyết định được ứng dụng rộng rãi trong nhiều lĩnh vực như tài chính, y tế, giáo dục và thương mại điện tử.



Hình 4: Minh họa một cây quyết định về nguy cơ mắc bệnh đau tim.

- Mặc dù dễ triển khai và có khả năng giải thích tốt, cây quyết định đơn lẻ thường dễ xảy ra hiện tượng quá khớp (Overfitting) khi dữ liệu phức tạp. Để khắc phục hạn chế này, nhiều phương pháp nâng cao đã được phát triển, trong đó nổi bật là XGBoost.

4.2.2. Thuật toán XGBoost (Extreme Gradient Boosting)

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy được phát triển dựa trên mô hình Gradient Boosting Decision Tree. XGBoost hiện là một trong những giải pháp mạnh mẽ và hiệu quả nhất cho các bài toán phân loại và hồi quy trên dữ liệu cấu trúc dạng bảng.

XGBoost sử dụng kỹ thuật Gradient Boosting. Mô hình không cố gắng xây dựng một cây quyết định lớn duy nhất mà kết hợp kết quả từ một chuỗi gồm nhiều cây quyết định yếu, trong đó cây sau sẽ tập trung tối ưu hóa và sửa chữa những sai sót mà các cây phía trước chưa giải quyết được. Kết quả dự đoán cuối cùng là tổng hợp có trọng số từ dự đoán của tất cả các cây cộng lại:

$$\text{Prediction} = \sum_{m=1}^M f_m(x) = \text{Tree}_1 + \text{Tree}_2 + \text{Tree}_3 + \dots + \text{Tree}_M$$

XGBoost còn có khả năng kiểm soát overfitting khá hiệu quả và thường cho kết quả ổn định hơn các thuật toán Gradient Boosting truyền thống.

4.2.3. Lý do lựa chọn XGBoost

Nhóm lựa chọn XGBoost làm mô hình nghiên cứu đại diện cho kỹ thuật Shallow Learning dựa trên các lý do sau:

- XGBoost đặc biệt phù hợp và cực kỳ mạnh mẽ với dữ liệu dạng bảng (tabular data). Bộ dữ liệu DoW được sử dụng trong đề án chính là dữ liệu dạng bảng!

- XGBoost cũng đã được chứng minh hiệu quả trong nhiều bài toán an ninh mạng. Nhiều nghiên cứu trước đây trong các lĩnh vực phát hiện tấn công DDoS, botnet, xâm nhập mạng và phát hiện gian lận đều ghi nhận XGBoost đạt độ chính xác cao và khả năng tổng quát hóa tốt trên dữ liệu thực tế.
- XGBoost có khả năng kiểm soát hiện tượng quá khớp (overfitting) khá hiệu quả. Đây là một đặc điểm quan trọng đối với bộ dữ liệu DoW, vốn tồn tại hiện tượng lặp dữ liệu và nguy cơ rò rỉ thông tin (Data Leakage) ở một số thuộc tính.
- XGBoost hỗ trợ đánh giá mức độ quan trọng của thuộc tính (Feature Importance). Tính năng này cho phép nhóm phân tích mức độ đóng góp của từng đặc trưng vào quá trình dự đoán, từ đó hỗ trợ lựa chọn thuộc tính, phát hiện các đặc trưng gây lệch mô hình và cải thiện khả năng giải thích kết quả nghiên cứu.

4.3. Lý thuyết về kiến trúc Transformer

4.3.1. Giới thiệu về Transformer

Transformer (ra đời năm 2017, trong bài báo *Attention is All You Need* huyền thoại) là kiến trúc học sâu loại bỏ hoàn toàn các mạng tuần hoàn (RNN) hoặc tích chập (CNN), hoạt động dựa trên cơ chế Attention (Chú ý).

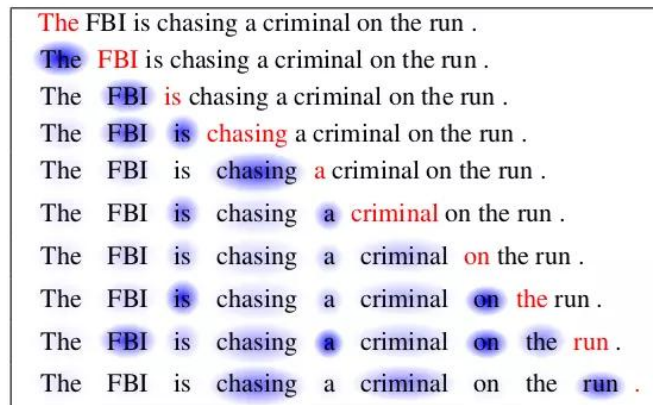
Kiến trúc này sở hữu hai ưu điểm cốt lõi:

- Xử lý song song: Cho phép tính toán toàn bộ chuỗi dữ liệu cùng một lúc, tối ưu hóa hiệu năng phần cứng và tăng tốc độ huấn luyện.
- Học các phụ thuộc xa: Khả năng liên kết và giữ ngữ cảnh giữa các từ/các đặc trưng ở khoảng cách xa trong một chuỗi rất hiệu quả.

Nhờ những đặc tính trên, Transformer đã trở thành nền tảng cốt lõi cho hầu hết các mô hình ngôn ngữ và học sâu hiện đại.

4.3.2. Cơ chế Self-Attention

Self-Attention (Tự chú ý) là thành phần cốt lõi giúp Transformer thiết lập mối quan hệ giữa các phần tử trong cùng một chuỗi dữ liệu mà không phụ thuộc vào khoảng cách. Thay vì xử lý tuần tự, mỗi phần tử trong chuỗi sẽ đồng thời "chú ý" vào tất cả các phần tử khác để tự cập nhật ngữ cảnh cho chính nó.



Hình 5: Minh họa cách Transformer hiểu và kết nối các từ trong câu.

Để lượng hóa mức độ "chú ý" này, Transformer sử dụng ba thành phần vectơ được biến đổi từ dữ liệu đầu vào thông qua công thức:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Trong đó:

- Q (Query - Truy vấn): Đại diện cho phần tử hiện tại đang xét, đóng vai trò truy vấn mối quan hệ với các thành phần khác trong toàn bộ chuỗi.
- K (Key - Khóa): Đại diện cho đặc trưng của tất cả các phần tử trong chuỗi dữ liệu. Phép nhân ma trận điểm tích vô hướng QK^T giúp xác định mức độ tương quan và độ quan trọng tương đối giữa Query và các Key tương ứng.
- V (Value - Giá trị): Chứa thông tin cốt lõi của dữ liệu đầu vào. Sau khi điểm tương quan đi qua hàm kích hoạt *softmax* để chuẩn hóa thành

phân phối xác suất (trọng số chú ý), ma trận này sẽ được nhân với V để trích xuất ra vector ngữ cảnh tổng hợp cuối cùng.

- d_k : Kích thước (số chiều) của vector thuộc tính Key. Việc chia cho $\sqrt{d_k}$ giúp thu nhỏ giá trị của phép nhân ma trận, tránh hiện tượng gradient bị triệt tiêu khi hàm *softmax* rơi vào vùng bão hòa khi số chiều quá lớn.

4.3.3. Multi-Head Attention

Kiến trúc Multi-Head Attention là bản nâng cấp mạnh mẽ dựa trên cơ chế Self-Attention đã trình bày. Ý tưởng cốt lõi là thay vì chỉ sử dụng một cơ chế Attention duy nhất (có thể chỉ tập trung vào một loại quan hệ nhất định giữa các phần tử), Transformer chia nhỏ các vector đại diện (Q, K, V) và cho chúng đi qua nhiều "đầu Attention" (heads) độc lập và song song. Mỗi Attention head này sẽ học cách "chú ý" vào các khía cạnh không gian ngữ cảnh khác nhau của dữ liệu. Kết quả đầu ra của các đầu Attention độc lập này sau đó sẽ được nối lại và chiếu qua một lớp tuyến tính cuối cùng để tạo ra một đại diện ngữ cảnh toàn diện, tổng hợp được nhiều loại quan hệ phức tạp trong dữ liệu đầu vào. Sự kết hợp này giúp Multi-Head Attention có khả năng nắm bắt ngữ cảnh tốt hơn và phong phú hơn so với việc chỉ sử dụng một đầu Attention.

Ví dụ, khi xử lý câu: *"Con mèo không ăn cá vì nó đã no."*

Từ "nó" có thể liên quan đến nhiều thành phần khác nhau trong câu. Một Attention Head có thể tập trung vào mối quan hệ ngữ pháp giữa "nó" và "con mèo", trong khi một Head khác có thể tập trung vào mối quan hệ ngữ nghĩa giữa "nó" và trạng thái "đã no". Đồng thời, một Head khác nữa có thể học cấu trúc tổng thể của câu để hiểu nguyên nhân dẫn đến hành động "không ăn cá".

4.3.4. Biểu diễn dữ liệu đầu vào dạng bảng bằng Embedding

Khi áp dụng Transformer vào dữ liệu dạng bảng (Tabular Data) phục vụ giám sát và phân tích an ninh mạng, thách thức lớn nhất là làm thế nào để chuyển đổi các thuộc tính định lượng và định danh thành một không gian biểu diễn chung mà không làm mất đi ngữ cảnh riêng biệt của từng thuộc tính.

- **Feature Embedding:** Kiến trúc Transformer không thể tiếp nhận trực tiếp các giá trị số hoặc các biến phân loại ở dạng thô từ cấu trúc bảng. Feature Embedding là quá trình chuyển đổi mỗi thuộc tính trong dữ liệu dạng bảng thành một vector số thực có kích thước cố định trước khi đưa vào Transformer. Nhờ đó, mô hình có thể học được các đặc trưng tiềm ẩn của từng thuộc tính và khai thác mối quan hệ giữa các thuộc tính thông qua cơ chế Self-Attention.
- **Kỹ thuật Tokenizer:** Trong dữ liệu dạng bảng, các thuộc tính khác nhau có thể mang cùng một giá trị nhưng lại có ý nghĩa hoàn toàn khác nhau. Ví dụ, giá trị $RTT = 50$ ms và $CPU = 50\%$ đều có giá trị số là 50 nhưng đại diện cho hai đặc trưng khác nhau. Để phân biệt các trường hợp này, kỹ thuật Tokenizer kết hợp định danh thuộc tính (Feature ID) và giá trị thuộc tính (Value) thành một token duy nhất trước khi thực hiện embedding.
- **Monotonic Embedding (Nhúng đơn điệu):** Việc rời rạc hóa dữ liệu có thể làm mất đi quan hệ thứ tự tự nhiên giữa các giá trị số, chẳng hạn $90 > 50 > 10$. Để khắc phục điều này, nhóm sử dụng kỹ thuật Monotonic Embedding nhằm bảo toàn thông tin định lượng của dữ liệu. Nhờ đó, các giá trị lớn hơn vẫn được biểu diễn theo hướng phản ánh đúng quan hệ thứ tự ban đầu.

4.3.5. Lý do lựa chọn Transformer

Nhóm lựa chọn Transformer làm mô hình nghiên cứu đại diện cho kỹ thuật Deep Learning dựa trên các lý do sau:

- Transformer có khả năng học các mối quan hệ dài hạn trong dữ liệu. Các cuộc tấn công Denial of Wallet thường được triển khai theo chiến lược *Low-and-Slow*, trong đó lưu lượng tấn công được phân tán trong thời gian dài nhằm tránh bị phát hiện. Cơ chế Self-Attention của Transformer cho phép mô hình học được các mối quan hệ xa giữa các đặc trưng và các sự kiện xuất hiện ở những thời điểm khác nhau.
- Transformer có khả năng phát hiện các mẫu bất thường phức tạp thông qua cơ chế Multi-Head Attention. Khi xem xét đồng thời nhiều đặc trưng như địa chỉ IP, thời gian truy cập, độ trễ mạng, thời gian thực thi hàm và mức sử dụng tài nguyên, Transformer có thể học được các mối quan hệ tiềm ẩn giữa chúng để phát hiện các hành vi bất thường mà các mô hình truyền thống có thể bỏ sót.
- Transformer hỗ trợ xử lý song song hiệu quả. Khác với các kiến trúc tuần tự như RNN hoặc LSTM, Transformer có thể xử lý toàn bộ dữ liệu đầu vào đồng thời thông qua cơ chế Attention. Điều này giúp rút ngắn đáng kể thời gian huấn luyện và tận dụng tốt hơn khả năng tính toán của phần cứng hiện đại.
- Nhiều công trình về phát hiện xâm nhập mạng (Network Intrusion Detection), botnet, DDoS và các hành vi bất thường trên hệ thống mạng đã ghi nhận Transformer đạt độ chính xác cao nhờ khả năng học các mối quan hệ phức tạp trong dữ liệu. Điều này cho thấy Transformer là một lựa chọn tiềm năng cho bài toán phát hiện tấn công Denial of Wallet trong môi trường serverless.

4.4. Tiền xử lý dữ liệu

4.4.1. Tổng quan về tập dữ liệu

Về bản chất hai tập dữ liệu đều giống nhau về hàng và cột, gồm 19 cột, với:

Tên thuộc tính	Kiểu dữ liệu	Mô tả
Id	Số nguyên (int64)	Mã định danh duy nhất của bản ghi dữ liệu
IP	Chuỗi (String)	Địa chỉ IP thực hiện yêu cầu tới hệ thống
bot	Boolean	Nhãn phân loại, xác định lưu lượng là bot tấn công (1) hay người dùng hợp lệ (0)
FunctionId	Số nguyên (int64)	Mã định danh của hàm Serverless được kích hoạt
functionTrigger	Chuỗi (String)	Loại sự kiện kích hoạt hàm (HTTP, Storage, SQL, ...)
timestamp	Chuỗi thời gian (String)	Thời điểm phát sinh yêu cầu
SubmitTime	Số nguyên (int64)	Thời điểm yêu cầu được gửi tới hệ thống
RTT	Số nguyên (int64)	Thời gian khứ hồi mạng (Round Trip Time)
InvocationDelay	Số thực (float64)	Độ trễ từ khi sự kiện xảy ra đến khi hàm bắt đầu thực thi
ResponseDelay	Số thực (float64)	Độ trễ từ khi hàm được gọi đến khi phản hồi được tạo ra

FunctionDuration	Số thực (float64)	Thời gian thực thi của hàm Serverless
ActiveFunctionsAt Request	Số nguyên (int64)	Số lượng hàm đang hoạt động tại thời điểm gửi yêu cầu
ActiveFunctionsAt Response	Số nguyên (int64)	Số lượng hàm đang hoạt động tại thời điểm nhận phản hồi
maxcpu	Số thực (float64)	Mức sử dụng CPU tối đa của máy chủ
avgcpu	Số thực (float64)	Mức sử dụng CPU trung bình
p95maxcpu	Số thực (float64)	Giá trị CPU tối đa tại phân vị 95%
vmcategory	Chuỗi (String)	Loại hoặc nhóm máy ảo được sử dụng
vmcorecountbucket	Số nguyên (int64)	Số lượng lõi CPU của máy ảo
vmmemorybucket	Số thực (float64)	Dung lượng bộ nhớ của máy ảo

Bảng 3: Chi tiết về các đặc trưng của tập dữ liệu

- Bộ dữ liệu đầu tiên có 187.087 bản ghi. Dữ liệu có sự mất cân bằng lớp trầm trọng khi số lượng mẫu tấn công chiếm khoảng 70,06% (131.072 mẫu), trong khi số lượng mẫu hợp lệ chiếm khoảng 29,94% (56.015 mẫu).
- Bộ dữ liệu thứ hai gồm 7 bộ dữ liệu nhỏ, mỗi bộ dữ liệu có 2 triệu bản ghi, với tỉ lệ mẫu tấn công lần lượt là 5%, 10%, 15%, 20%, 25%, 30%

và 35%. Nhóm kết hợp cả 7 tập dữ liệu này thành 1 tập dữ liệu lớn duy nhất có 14 triệu bản ghi.

4.4.2. Encoding & Feature Extraction

Vì 2 bộ dữ liệu giống nhau về số cột và kiểu dữ liệu các cột, nhóm xử lý cả hai theo cùng một cách:

- Mục tiêu chính là biến đổi dữ liệu thô thành một tập dữ liệu có cấu trúc dạng số và danh mục chuẩn hóa để sẵn sàng đưa vào các mô hình.
- *add_ip_rolling_features*: Hàm này tính toán tần suất xuất hiện của một địa chỉ IP trong một khoảng thời gian nhất định (Kỹ thuật cửa sổ trượt - Rolling Window). Ở đây nhóm tính toán thời gian xuất hiện của IP đó trong 1 phút/1 tiếng trước.
- *engineering_features*: Hàm này sẽ:
 - + Trích xuất ra 2 đặc trưng mới, giờ trong ngày (*hourOfDay*) và ngày trong tuần (*dayOfWeek*). Điều này rất phù hợp với bản chất của tập dữ liệu và bản chất của cuộc tấn công.
 - + Chuyển cột *FunctionId* sang kiểu dữ liệu danh mục (category) để tránh Model đưa ra những quy tắc sai lệch với cột này, giống như $FunctionId > 5 \Rightarrow Bot$.
 - + One-Hot Encoding: Biến đổi các giá trị chữ trong cột *functionTrigger* và *vmcategory* thành các cột nhị phân (0 hoặc 1).
 - + Gộp các cột One-Hot Encoding mới vào bảng chính, đồng thời xóa bỏ các cột dữ liệu gốc (IP, timestamp, functionTrigger, vmcategory) vì chúng đã được trích xuất hoặc biến đổi xong, không còn cần thiết cho mô hình nữa.

Kết quả là ta nhận được một tập dữ liệu dạng bảng hoàn toàn ở dạng số và đã được chuẩn hóa theo thời gian.

4.4.3. EDA - Exploratory Data Analysis

Mục tiêu: Tự động phân loại một cột số là dữ liệu rời rạc (Discrete) hay dữ liệu liên tục (Continuous) dựa trên việc phân tích cấu trúc phân vị (Quantiles). Sau đó, toàn bộ báo cáo thống kê sẽ được xuất ra một file định dạng JSON.

- *build_quantiles*: Hàm này tính toán các giá trị phân vị từ 5%, 10%, 15%... cho đến 95% và phân vị 99% của một cột số.
- *build_quantile_profile*: Phân tích các mốc phân vị vừa tính được để xem chúng biến thiên như thế nào.
- *classify_numeric_by_quantiles*: Hàm phân loại chính.

Một cột số sẽ được coi là Dữ liệu liên tục (Continuous) nếu nó thỏa mãn một trong hai điều kiện:

- + Có hơn 15% số **phân vị** là số thập phân lẻ (*has_fractional_quantiles*).
- + Các giá trị phân vị thay đổi rất đa dạng (tỷ lệ độc nhất $\geq 85\%$) VÀ cột đó có hơn 100 giá trị khác nhau hoàn toàn (*high_cardinality*).

Ngược lại nếu không thỏa mãn, cột số đó sẽ bị xếp vào Dữ liệu rời rạc (Discrete).

Trong hàm xử lý EDA chính (*network_domain_eda*), nó còn tính toán thêm các thông số cơ bản: số lượng dòng (*count*), số lượng ô trống (*missing_count*), kiểu dữ liệu (*data_type*), trung bình (*mean*), độ lệch chuẩn (*std*), min, max, số lượng giá trị độc nhất (*unique_count*) và giá trị xuất hiện nhiều nhất chiếm bao nhiêu % trong toàn bộ cột (*top_frequency_percent*).

4.4.4. Chuẩn hóa và biến đổi dữ liệu

Nhiệm vụ của nó là đọc file báo cáo JSON đã sinh ra ở bước trước để áp dụng các kỹ thuật biến đổi toán học nhằm ép tất cả dữ liệu (trừ các cột One-hot) về cùng một khoảng giá trị mục tiêu từ 0 -> 20,000.

- Biến đổi dữ liệu thời gian (Cyclical Encoding): Các cột chỉ thời gian có tính chất tuần hoàn. Nếu để số tự nhiên, mô hình sẽ hiểu số 23 (23h) và số 0 (0h) rất xa nhau, dù thực tế chúng chỉ cách nhau 1 tiếng. Nhóm ánh xạ thời gian lên một vòng tròn lượng giác bằng Sin và Cos, sau đó cộng 1 (để ép khoảng từ $[-1, 1]$ thành $[0, 2]$) và nhân 10000 để ép kết quả vào khoảng $[0, 20000]$.
- Xử lý dữ liệu rời rạc: Đối với các cột được định nghĩa là rời rạc trong file JSON, nhóm sử dụng kỹ thuật Piecewise Linear qua hàm *QuantileTransformer* của thư viện *sklearn*.

Hàm này làm phẳng phân phối của dữ liệu rời rạc về dạng phân phối đồng đều trong khoảng $[0, 1]$ và giúp giảm bớt ảnh hưởng của các giá trị ngoại lai (outliers).

Cuối cùng, nhóm nhân toàn bộ kết quả với 20000 (scale).

- Xử lý dữ liệu liên tục: Đối với các cột liên tục, nhóm sử dụng phương pháp chia tỉ lệ dựa trên giá trị lớn nhất thu được từ file báo cáo EDA trước đó. Công thức là (cộng 0.1 để vừa tránh chia cho 0, vừa đảm bảo giá trị luôn < 20000)

$$Value_{new} = Value_{old} \times \frac{20000}{Max+0.1}$$

Toàn bộ dữ liệu của cột liên tục sẽ được co giãn tuyến tính về khoảng $[0, 20000]$. Điều này giúp giảm ảnh hưởng của các giá trị nhiễu (thứ vốn không thể tránh khỏi trong dữ liệu mạng) lên sự chính xác của mô hình.

4.4.5. Luồng xử lý tổng thể

Toàn bộ các bước trên được đóng gói thành một Pipeline xử lý tuần tự từ dữ liệu thô đến một tập dữ liệu gồm 28 cột, sẵn sàng cho mô hình huấn luyện.

- Bước 1: Đọc dữ liệu từ các tệp CSV gốc.

- Bước 2: Tiến hành làm giàu dữ liệu bằng cách tính toán cửa sổ trượt IP và mã hóa One-Hot các biến kiểu chữ. Dữ liệu lúc này được chuyển hoàn toàn về dạng số.
- Bước 3: Chạy đoạn code EDA tự động để quét qua các cột số, phân tích hồ sơ phân vị và kết xuất cấu trúc của bộ dữ liệu thành tệp định dạng *eda_report.json*. Tệp này đóng vai trò là "bộ cấu hình" cho giai đoạn sau, đồng thời cho nhóm hướng đi cho các bước kế tiếp.
- Bước 4: Đọc tệp cấu hình JSON, ánh xạ các quy tắc chuẩn hóa tương ứng cho từng loại thuộc tính để ép toàn bộ không gian dữ liệu về thang đo đồng nhất [0, 20000].

4.5. Huấn luyện mô hình XGBoost

Nhóm sử dụng thư viện *XGBoost* có sẵn:

```
# 2. Khởi tạo và Train
xgb_model = xgb.XGBClassifier(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    scale_pos_weight=ratio,
    random_state=42,
    n_jobs=-1,
    enable_categorical=True,
)
xgb_model.fit(X_train_ml, y_train_ml)
```

Hình 5: Đoạn code khởi tạo mô hình XGBoost

Để giải quyết bài toán mất cân bằng lớp thực tế từ dữ liệu, nhóm tích hợp tham số tính toán trọng số động. Cơ chế này giúp mô hình gán mức độ ưu tiên cao hơn cho các lớp có ít mẫu, từ đó giảm xu hướng thiên lệch về lớp chiếm đa số và cải thiện khả năng nhận diện các lớp thiểu số trong quá trình huấn luyện.

$$\text{scale_pos_weight} = \frac{\sum(y = 0)}{\sum(y = 1)}$$

Ý nghĩa các siêu tham số (Hyperparameters):

- *n_estimators=100*: Số lượng cây quyết định (Decision Trees) được huấn luyện tuần tự. Giá trị 100 được lựa chọn nhằm cân bằng giữa năng lực học của mô hình và chi phí tính toán thực nghiệm.
- *max_depth=6*: Chiều sâu tối đa của mỗi cây. Giới hạn độ sâu bằng 6 giúp kiểm soát tốt hiện tượng overfitting, đảm bảo các quy tắc phân tách của cây không quá lỏng lẻo nhưng cũng không quá chi tiết.
- *learning_rate=0.1*: Tốc độ học, thu nhỏ đóng góp của mỗi cây mới được thêm vào với hệ số 0.1, giúp quá trình hội tụ mượt mà và chính xác hơn.
- *n_jobs=-1*: Tận dụng tối đa tất cả các lõi CPU có sẵn trên máy tính để song song hóa quá trình tính toán, tăng tốc thời gian huấn luyện.
- *enable_categorical=True*: Cho phép XGBoost nhận diện và xử lý trực tiếp các cột có kiểu dữ liệu category (như cột FunctionId ở các bước trước).

Sau khi huấn luyện hoàn tất, nhóm sử dụng Gain Score để đánh giá mức độ đóng góp của từng thuộc tính vào quá trình ra quyết định của mô hình. Từ đó, nhóm có thể xác định các thuộc tính có ảnh hưởng quá lớn và thực hiện tinh chỉnh tập dữ liệu nhằm nâng cao hiệu quả dự đoán.

4.6. Huấn luyện mô hình Transformer

4.6.1. Token hóa dữ liệu bảng bằng nhúng đơn điệu

Thông thường, Transformer xử lý một câu bằng cách biến mỗi từ thành một Vector (gọi là kỹ thuật Token Embedding). Đối với dữ liệu bảng, để đưa vào Transformer, nhóm biến mỗi ô trong bảng thành một Vector Token, sử dụng kỹ thuật nhúng đơn điệu (Monotonic Embedding). Kỹ thuật nhúng đơn điệu được áp dụng nhằm bảo toàn đặc tính thứ bậc và độ lớn liên tục của dữ liệu số dạng bảng (như mức CPU, độ trễ mạng), khắc phục hoàn

toàn nhược điểm mất nhận thức định lượng của tầng nhúng Transformer truyền thống.

- Khởi tạo (`__init__`)

```
def __init__(self, num_features, vocab_size, embedding_dim):
    super().__init__()
    # Cần ít nhất 1 chiều cho tọa độ đơn điệu, và 1 chiều cho phần embedding có thể học được
    if embedding_dim < 2:
        raise ValueError("embedding_dim must be >= 2 for monotonic + trainable coordinates")

    self.num_features = num_features
    self.vocab_size = vocab_size
    self.embedding_dim = embedding_dim
    # Bảng embedding lưu vector riêng cho từng cặp
    # (feature_id, value_id)
    self.cell_embed = nn.Embedding(
        num_embeddings=num_features * vocab_size,
        embedding_dim=embedding_dim - 1,
    )
    # Lưu danh sách feature id để tái sử dụng khi forward
    self.register_buffer('feature_ids', torch.arange(num_features, dtype=torch.long))
```

Hình 6: Đoạn code khởi tạo bảng tra cứu Vector khả học và các tham số cần thiết

Tạo 1 bảng tra cứu chứa các vector khả học, với số chiều bớt đi một (so với số chiều mục tiêu, vì ta cần dành 1 chiều cho tọa độ đơn điệu).

- Định nghĩa Forward Propagation của dữ liệu:

```
def forward(self, x):
    """
    x: Tensor [batch_size, num_features]
        chứa các giá trị rời rạc đã được mã hóa số.

    Trả về:
        Tensor [batch_size, num_features, embedding_dim]
    """
    x = x.long().clamp(min=0, max=self.vocab_size - 1)

    # Tạo id duy nhất cho mỗi cặp (feature, value). Điều này giúp mỗi ô dữ liệu có embedding riêng biệt.
    cell_ids = self.feature_ids.unsqueeze(0) * self.vocab_size + x
    # Phần embedding thông thường
    trainable_part = self.cell_embed(cell_ids)

    # Tọa độ đơn điệu: giá trị tỉ lệ càng lớn thì tọa độ thứ nhất càng lớn.
    monotonic_part = (x.float() / (self.vocab_size - 1)).unsqueeze(-1)

    # Ghép: [tọa độ đơn điệu | embedding học được]
    return torch.cat([monotonic_part, trainable_part], dim=-1)
```

Hình 7: Đoạn code về phần Forward Propagation

- + Dùng kỹ thuật tuyến tính hóa mảng nhiều chiều với công thức $ID = \text{Số thứ tự cột} * \text{vocab_size} + \text{Giá trị hiện tại}$ giúp mỗi giá trị trong bảng có một định danh độc nhất, các giá trị giống nhau nhưng xuất hiện ở các cột khác nhau được ánh xạ tới các vector nhúng khác nhau.
- + Tra cứu Vector học được: Máy tính mapping ID của giá trị hiện tại với giá trị trong bảng *self.cell_embed*. Bảng này sẽ trả về các Vector chứa các số thực có số chiều là *embedding_dim - 1*. Những con số này ban đầu là ngẫu nhiên, nhưng qua quá trình AI học tập, các con số này sẽ tự động thay đổi để thể hiện mối quan hệ ngữ nghĩa ngầm giữa các ô dữ liệu.
- + Tạo tọa độ đơn điệu: Chuyển giá trị nguyên gốc sang số thực rồi chia cho giá trị lớn nhất có thể đạt được (*vocab_size - 1*). Phép tính này ép mọi giá trị trong bảng về một tỷ lệ nằm nghiêm ngặt trong đoạn $[0.0, 1.0]$, và thêm một chiều dữ liệu vào cuối để biến con số tỷ lệ đơn lẻ này thành một vector có độ dài bằng 1 (Ví dụ từ số 0.4 biến thành mảng $[0.4]$). Thành phần này đóng vai trò bảo toàn mối quan hệ thứ tự cũng như độ lớn tương đối giữa các giá trị trong dữ liệu.
- + Cuối cùng, PyTorch sẽ ghép nối lại Vector biểu diễn giá trị đó theo công thức $[\text{tọa độ đơn điệu} \mid \text{embedding học được}]$.

Kích thước đầu ra cuối cùng của tập dữ liệu sẽ là $[\text{batch_size}, \text{num_features}, \text{embedding_dim}]$. Mọi ô dữ liệu giờ đây đã chuyển hóa thành một Vector Token hoàn chỉnh, vừa giữ được tính chất độ lớn toán học, vừa giữ được ngữ nghĩa học sâu.

4.6.2. Transformer Encoder

Sau khi dữ liệu bảng được chuyển đổi thành các vector nhúng thông qua khối Token hóa phía trên, toàn bộ không gian dữ liệu sẽ được chuyển tiếp

vào khối Transformer Encoder. Đây là cấu phần cốt lõi chịu trách nhiệm khai phá và trích xuất mối quan hệ phụ thuộc phi tuyến tính giữa các đặc trưng trong cùng một bản ghi hệ thống.

Khối này được cấu thành từ hai tầng xử lý tuần tự:

- **Multi-Head Attention:** Khác với các mô hình xử lý thuộc tính theo hướng độc lập, cơ chế Self-Attention cho phép mỗi đặc trưng tương tác với toàn bộ các đặc trưng còn lại trong cùng bản ghi dữ liệu. Với cơ chế nhiều attention heads, mô hình có thể đồng thời học nhiều dạng tương quan khác nhau, chẳng hạn giữa mức sử dụng tài nguyên hệ thống, độ trễ mạng và tần suất truy cập IP. Sau bước này, mỗi vector đặc trưng đã được cập nhật dựa trên các vector đặc trưng khác.
- **Feed-Forward Network:** Sau tầng Self-Attention, mỗi vector đặc trưng được xử lý bởi một mạng nơ-ron truyền thẳng nhằm thực hiện biến đổi phi tuyến và tăng cường khả năng biểu diễn của mô hình.

```
# Module tokenizer chịu trách nhiệm chuyển dữ liệu bảng từ dạng số nguyên sang vector embedding.
self.tokenizer = tokenizer

# 1. Transformer Encoder học quan hệ giữa các cell/feature trong cùng một dòng.
encoder_layer = nn.TransformerEncoderLayer(
    d_model=embedding_dim,
    nhead=4,
    dim_feedforward=256,
    dropout=0.1,
    batch_first=True,
)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2)
```

Hình 8: Đoạn code về khởi tạo Transformer Encoder

Ý nghĩa các tham số:

- *d_model=embedding_dim*: Đây là kích thước đầu vào và đầu ra của khối Transformer Encoder. Giá trị này quyết định lượng thông tin mà mỗi đặc trưng có thể mang theo trong quá trình học.
- *nhead=4*: Số lượng Attention Heads được sử dụng trong cơ chế Multi-Head Attention. Việc sử dụng 4 đầu chú ý cho phép mô hình đồng thời học nhiều dạng tương quan khác nhau giữa các thuộc tính, từ đó khai

thác được các mối quan hệ phức tạp trong dữ liệu thay vì chỉ tập trung vào một góc nhìn duy nhất.

- *dim_feedforward=256*: Kích thước lớp ẩn của mạng Feed-Forward Network bên trong mỗi Encoder Layer. Tham số này quy định năng lực biểu diễn phi tuyến của mô hình.
- *dropout=0.1*: Tỷ lệ loại bỏ ngẫu nhiên các nút trong quá trình huấn luyện. Cơ chế này giúp giảm hiện tượng quá khớp (Overfitting), tăng khả năng tổng quát hóa của mô hình trên các tập dữ liệu chưa từng xuất hiện.

4.6.3. Làm phẳng (Flatten)

Dữ liệu sau khi đi qua khối Transformer Encoder vẫn duy trì cấu trúc Tensor 3 chiều với kích thước $[batch_size, num_features, embedding_dim]$, biểu diễn dưới dạng một danh sách các vector đặc trưng song song. Tuy nhiên, khối phân loại kế tiếp yêu cầu định dạng đầu vào của mỗi mẫu dữ liệu phải là một vector một chiều liên tục. Để giải quyết sự bất tương thích về mặt cấu trúc này, nhóm triển khai khối *nn.Flatten()* nhằm duỗi phẳng không gian ma trận, với kích thước 1D khi này sẽ là $num_features * embedding_dim$. Tiến trình này thực hiện nối kết tuần tự các vector đặc trưng thành một hàng ngang duy nhất.

```
# Transformer trả về: [batch_size, num_features, embedding_dim]
# Flatten biến đổi thành: [batch_size, num_features * embedding_dim] để đưa vào mạng phân loại phía sau.
self.flatten = nn.Flatten()

input_dim = num_features * embedding_dim
```

Hình 9: Gọi hàm *Flatten()* của *PyTorch*

Nhóm lựa chọn Flatten nhằm bảo toàn toàn bộ thông tin từ các đặc trưng sau bước Attention thay vì thực hiện phép tổng hợp thông tin như Mean Pooling.

4.6.4. Mạng nơ-ron phân loại nhị phân (Binary Classifier)

Sau khi được xử lý bởi Transformer Encoder và được làm phẳng (Flatten), toàn bộ các vector đặc trưng được đưa vào một mạng nơ-ron truyền thẳng (MLP) để thực hiện phân loại nhị phân.

Mạng MLP gồm một tầng ẩn với 128 nút có nhiệm vụ tổng hợp và trích xuất các đặc trưng quan trọng từ biểu diễn đầu ra của Transformer. Tiếp theo, hàm kích hoạt ReLU được sử dụng để bổ sung tính phi tuyến, giúp mô hình học các ranh giới phân loại phức tạp giữa lưu lượng hợp lệ và lưu lượng bot.

Để hạn chế hiện tượng quá khớp, tầng Dropout được áp dụng, ngẫu nhiên loại bỏ 20% số nút trong quá trình huấn luyện. Cuối cùng, tầng Linear ánh xạ biểu diễn đặc trưng sang một giá trị logit duy nhất. Giá trị logit sau đó được chuyển đổi thành xác suất thông qua hàm Sigmoid trong giai đoạn suy luận, làm cơ sở cho việc dự đoán nhãn Bot (1) hoặc Normal Traffic (0).

```
# 3. Binary classifier.
# MLP chịu trách nhiệm tổng hợp toàn bộ thông tin đã học từ Transformer và đưa ra quyết định cuối cùng.
self.classifier = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 1),
)
```

Hình 10: Đoạn code triển khai Binary Classifier

4.6.5. Kiến trúc tổng thể mô hình

Quá trình lan truyền xuôi (Forward Propagation) của mô hình được thực hiện tuần tự qua bốn giai đoạn chính. Đầu tiên, dữ liệu đầu vào được chuyển đổi thành các vector nhúng thông qua khối Monotonic Tokenizer. Tiếp theo, các vector này được đưa vào khối Transformer Encoder để học các mối quan hệ và tương tác giữa các đặc trưng trong cùng một bản ghi dữ liệu.

Sau đó, đầu ra của Transformer được làm phẳng thành vector một chiều nhằm phù hợp với định dạng đầu vào của bộ phân loại. Cuối cùng, vector đặc trưng được chuyển vào MLP Binary Classifier để sinh ra một giá trị logit duy nhất, làm cơ sở cho việc dự đoán nhãn Bot hoặc lưu lượng hợp lệ.

Luồng xử lý tổng quát của mô hình được biểu diễn như sau:

Input → Monotonic Tokenizer → Transformer Encoder → Flatten → MLP Classifier → Output

Mô hình được viết thành 1 Class tên là *DoW_Classifier* để dễ triển khai sau này.

```
def forward(self, x):
    embedded = self.tokenizer(x)
    attended = self.transformer(embedded)
    flat = self.flatten(attended)
    out = self.classifier(flat)
    return out
```

Hình 11: Triển khai tổng thể mô hình trong Class *DoW_Classifier*

4.6.6. Huấn luyện mô hình

- Đầu tiên, dữ liệu sau tiền xử lý được chuyển đổi từ định dạng NumPy sang Tensor của PyTorch. Dữ liệu được đóng gói ở dạng TensorDataset và được lưu dưới RAM, chỉ đẩy lên VRAM của GPU khi cần.
- Kích thước Embedding nhóm chọn là 64, nghĩa là mỗi ô dữ liệu sẽ được nhúng thành 1 Vector 64 chiều.
- Vocabulary Size sẽ là giá trị số lớn nhất trong tập Train và tập Test cộng thêm một, đảm bảo bao quát toàn bộ miền giá trị của dữ liệu.
- Tiếp theo, nhóm sử dụng DataLoader để chia dữ liệu thành các lô nhỏ, mỗi lô có BATCH_SIZE = 4096. Nghĩa là mỗi lần huấn luyện, model sẽ xử lý 4096 mẫu dữ liệu cùng lúc. Tiến trình tích hợp cấu hình *num_workers = 4* nhằm tận dụng xử lý đa luồng từ CPU để chuẩn bị

trước dữ liệu, đồng thời kích hoạt *pin_memory = True* phối hợp với tham số *non_blocking = True* để tăng tốc độ đẩy dữ liệu từ RAM lên VRAM GPU thông qua kiến trúc truyền tải bất đồng bộ.

- Nhóm sử dụng bộ tối ưu hóa Adam Optimizer với tốc độ học khởi tạo *lr = 0.001*. Để kiểm soát bài toán mất cân bằng lớp, nhóm triển khai hàm mất mát *BCEWithLogitsLoss* kết hợp với trọng số động xác định bởi tỷ lệ giữa tổng mẫu hợp lệ trên tổng mẫu tấn công. Trọng số này được giúp tăng mức phạt đối với các trường hợp dự đoán sai ở lớp thiểu số. Nhờ đó, mô hình giảm xu hướng thiên lệch về lớp chiếm đa số, cải thiện khả năng phát hiện các mẫu tấn công và nâng cao hiệu quả học trên bộ dữ liệu mất cân bằng.

$$\text{pos_weight} = \frac{\sum(y = 0)}{\sum(y = 1)}$$

- Mô hình được thiết lập ở chế độ huấn luyện (*model.train()*) và thực thi lặp lại qua **5 Epochs**. Tại mỗi lượt truyền (Iteration), các batch dữ liệu sau khi nạp lên GPU sẽ lần lượt trải qua các bước: xóa bộ nhớ đệm đạo hàm (*optimizer.zero_grad()*), lan truyền tiến để đưa ra dự đoán, tính toán sai số phạt (Loss), lan truyền ngược (*loss.backward()*) để tính toán gradient và cập nhật hệ thống trọng số mạng (*optimizer.step()*). Lượng sai số trung bình (*avg_loss*) được theo dõi liên tục sau mỗi chu kỳ để đánh giá mức độ hội tụ toán học của mô hình.


```

# =====
# KHỞI TẠO MODEL MONOLITHIC EMBEDDING
# =====
NUM_FEATURES = X_train_dl.shape[1]
EMBED_DIM = 64

# Lấy max của cả train/test để tránh lỗi embedding index nếu test có value lớn hơn train.
VOCAB_SIZE = int(max(X_train_dl.max(), X_test_dl.max())) + 1
VOCAB_SIZE = max(VOCAB_SIZE, 2)

print(f"Setup Model: {NUM_FEATURES} features, Vocab size: {VOCAB_SIZE}, Embedding dim: {EMBED_DIM}")
print(f"Monolithic embedding table size: {NUM_FEATURES * VOCAB_SIZE:,} tokens")

tokenizer = MonolithicTabularTokenizer(
    num_features=NUM_FEATURES,
    vocab_size=VOCAB_SIZE,
    embedding_dim=EMBED_DIM,
)
model = DoW_Classifier(tokenizer, num_features=NUM_FEATURES, embedding_dim=EMBED_DIM).to(device)

# Tự động đếm số lượng hai lớp ngay trên tập Train.
num_positive = (y_train_dl == 1).sum()
num_negative = (y_train_dl == 0).sum()

# Tính trọng số động.
dynamic_pos_weight = num_negative / num_positive
print(f"Trọng số pos_weight tự động: {dynamic_pos_weight:.2f}")

weight_tensor = torch.tensor([dynamic_pos_weight], dtype=torch.float32).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=weight_tensor)
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Hình 12: Khởi tạo mô hình từ Class `DoW_Classifier` xây dựng phía trên

Chương 5. ĐÁNH GIÁ MÔ HÌNH

5.1. Kết quả thực nghiệm và đánh giá

5.1.1. Chia tập Train/Test

Dữ liệu được chia thành tập huấn luyện và tập kiểm thử theo tỷ lệ 80:20 bằng hàm `train_test_split()` của thư viện *Scikit-learn*. Nhóm áp dụng kỹ thuật phân tầng (stratify) theo nhãn bot để duy trì phân bố lớp giữa hai tập dữ liệu, đồng thời cố định `random_state = 42` nhằm đảm bảo tính nhất quán và khả năng tái lập kết quả thực nghiệm.

```
# CHIA TRAIN/TEST
print("Đang chia cắt tập Train và tập Test...")

FUNCTION_ID_CATEGORIES = None
if "FunctionId" in df.columns:
    FUNCTION_ID_CATEGORIES = sorted(df["FunctionId"].dropna().unique())

df, df_test = train_test_split(
    df,
    test_size=0.2,
    shuffle=True,
    stratify=df["bot"],
    random_state=42,
)
```

Hình 13: Code chia tập dữ liệu thành 2 tập Train và Test

5.1.2. Đánh giá mô hình trên dataset thử nghiệm

Đây là Dataset từ bài báo *Generation of a dataset for DoW attack detection in serverless architectures* ở [phần 2.4](#). Như nhóm đã nhắc tới phía trên, dataset này có rất nhiều nhược điểm, nên nhóm chỉ dùng để test thuật toán. Khi nhóm huấn luyện mô hình trên dataset này, có rất nhiều vấn đề lộ ra:

- Data Leakage rất nặng! Điểm Gain Score từ mô hình XGBoost của đặc trưng `dayOfWeek_sin` có số điểm lên tới 1.0, nghĩa là model gần như trở thành 1 câu lệnh rẽ nhánh *if-else* theo đặc trưng ngày tháng!

Bảng trọng số các cột ảnh hưởng nhất:

	Feature	Gain	Score
25	dayOfWeek_sin		1.0
1	SubmitTime		0.0
0	FunctionId		0.0
3	InvocationDelay		0.0
4	ResponseDelay		0.0
5	FunctionDuration		0.0
6	ActiveFunctionsAtRequest		0.0
7	ActiveFunctionsAtResponse		0.0
8	maxcpu		0.0
9	avgcpu		0.0
2	RTT		0.0
10	p95maxcpu		0.0
11	vmcorecountbucket		0.0
13	ipMin		0.0
12	vmmemorybucket		0.0
15	functionTrigger_http		0.0
16	functionTrigger_notification		0.0
17	functionTrigger_sql		0.0
14	ipHour		0.0
18	functionTrigger_storage		0.0
19	functionTrigger_stream		0.0
21	vmcategory_Interactive		0.0
20	vmcategory_Delay-insensitive		0.0
22	vmcategory_Unkown		0.0
23	hourOfDay_sin		0.0
24	hourOfDay_cos		0.0
26	dayOfWeek_cos		0.0

Hình 14: Bảng Gain Score của XGBoost, với đặc trưng *dayOfWeek_sin* có điểm 1.0

Khi nhóm thử drop cột *dayOfWeek_sin*, sẽ có các khác như cột *dayOfWeek_cos*, *ipHour*, *ipMin*, *hourOfDay_sin*... lần lượt dẫn đầu điểm Gain Score với số điểm 1.0 hoặc sát với 1.0, drop cột này sẽ lại có cột khác, và điểm chung là đều liên quan tới cột IP hoặc thời gian gốc

=> Dataset này bị Data Leakage rất nặng nề, mô hình huấn luyện từ đây sẽ cực kỳ phụ thuộc vào các chỉ số IP và thời gian, và gần như không thể sử dụng trong thực tế.

- Mô hình XGBoost và Transformer đều cho kết quả không tưởng khi huấn luyện trên tập dữ liệu này: Accuracy 100%!

[XGBoost] Thời gian Inference: 0.0531 giây					
--- BÁO CÁO KẾT QUẢ ---					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	11203	
1	1.00	1.00	1.00	26215	
accuracy			1.00	37418	
macro avg	1.00	1.00	1.00	37418	
weighted avg	1.00	1.00	1.00	37418	
[Transformer] Thời gian Inference: 0.3918 giây					
--- BÁO CÁO KẾT QUẢ ---					
	precision	recall	f1-score	support	
0.0	1.00	1.00	1.00	11203	
1.0	1.00	1.00	1.00	26215	
accuracy			1.00	37418	
macro avg	1.00	1.00	1.00	37418	
weighted avg	1.00	1.00	1.00	37418	

Hình 15: Kết quả suy luận của 2 mô hình đạt Accuracy 100%

- Khi nhóm thử loại bỏ hết tất cả các cột liên quan tới IP và thời gian, hiệu quả của cả hai mô hình tụt xuống thê thảm và rất kém:

[XGBoost] Thời gian Inference: 0.0329 giây					
--- BÁO CÁO KẾT QUẢ ---					
	precision	recall	f1-score	support	
0	0.39	0.87	0.54	11203	
1	0.89	0.42	0.57	26215	
accuracy			0.56	37418	
macro avg	0.64	0.65	0.56	37418	
weighted avg	0.74	0.56	0.56	37418	
[Transformer] Thời gian Inference: 0.6715 giây					
--- BÁO CÁO KẾT QUẢ ---					
	precision	recall	f1-score	support	
0.0	0.35	0.99	0.51	11203	
1.0	0.99	0.20	0.33	26215	
accuracy			0.44	37418	
macro avg	0.67	0.60	0.42	37418	
weighted avg	0.80	0.44	0.39	37418	

Hình 16: Kết quả suy luận tụt dốc thảm hại

- + XGBoost: Class 0 (User) có recall cao (0.87) → model nhận diện tốt người dùng thật, nhưng precision thấp (0.39) → rất nhiều Bot bị

gán nhầm thành User. Class 1 (Bot) có precision cao (0.89), nhưng recall thấp (0.42) → mô hình bỏ sót một lượng lớn Bot thực tế.

- + Transformer: Class 0 có recall gần như tuyệt đối (0.99) → nhận diện rất tốt người dùng thật. Nhưng precision cực thấp (0.35) → rất nhiều Bot bị bỏ lọt, False Negative cực kỳ cao. Class 1 có precision gần như tuyệt đối (0.99) → khi dự đoán là thì hầu như đúng, nhưng recall cực thấp (0.20) → bỏ lỡ tới 80% các traffic tấn công thật!

=> Những điều này chứng tỏ các mô hình trước đó không thực sự học được đặc trưng hành vi của tấn công Denial of Wallet mà chủ yếu khai thác các quy luật nhân tạo tồn tại trong dữ liệu mô phỏng. Do đó, bộ dữ liệu DoWTS này chỉ phù hợp để đánh giá sơ bộ thuật toán nhưng chưa đủ độ tin cậy để phản ánh môi trường vận hành thực tế.

5.1.3. Đánh giá mô hình trên Dataset huấn luyện chính

Đây là Dataset từ bài báo *Data augmentation with Generative AI for DoW attack detection in serverless architectures* [ở phần 2.5](#), là dataset huấn luyện chính của nhóm.

Trong quá trình thử nghiệm, nhóm nhận thấy bộ dữ liệu tồn tại hiện tượng Data Leakage đáng kể ở các thuộc tính liên quan đến thời gian. Tuy nhiên, các đặc trưng thời gian cũng là một phần quan trọng trong bối cảnh hoạt động của hệ thống serverless và trong kịch bản mô phỏng tấn công do tác giả đề xuất. Vì vậy, thay vì loại bỏ toàn bộ các thuộc tính thời gian, nhóm chỉ loại bỏ hai thuộc tính *dayOfWeek_sin* và *dayOfWeek_cos*, là những thuộc tính thể hiện mức độ tương quan quá mạnh với nhãn dữ liệu. Cách tiếp cận này giúp hạn chế ảnh hưởng của Data Leakage trong khi vẫn bảo toàn được ngữ cảnh thời gian cần thiết cho quá trình huấn luyện và đánh giá mô hình.

```

--- ĐANG HUẤN LUYỆN XGBOOST ---
Bảng trọng số các cột ảnh hưởng nhất:

```

	Feature	Gain	Score
24	hourOfDay_cos	0.791883	
13	ipMin	0.094714	
23	hourOfDay_sin	0.062750	
20	vmcategory_Delay-insensitive	0.016862	
18	functionTrigger_storage	0.003338	
0	FunctionId	0.003331	
6	ActiveFunctionsAtRequest	0.003125	
22	vmcategory_Unkown	0.003108	
16	functionTrigger_notification	0.002526	
15	functionTrigger_http	0.002510	
12	vmmemorybucket	0.002095	
9	avgcpu	0.001875	
2	RTT	0.001794	
10	p95maxcpu	0.001439	
1	SubmitTime	0.001315	
14	ipHour	0.001281	
4	ResponseDelay	0.001163	
3	InvocationDelay	0.001000	
19	functionTrigger_stream	0.000889	
21	vmcategory_Interactive	0.000762	
8	maxcpu	0.000696	
11	vmcorecountbucket	0.000573	
5	FunctionDuration	0.000511	
7	ActiveFunctionsAtResponse	0.000373	
17	functionTrigger_sql	0.000086	

Hình 17: Bảng Gain Score của XGBoost, cải thiện đáng kể tình trạng Data Leakage

Sau quá trình huấn luyện và đánh giá, cả hai mô hình XGBoost và Transformer đều đạt hiệu năng rất cao trên tập kiểm thử với độ chính xác xấp xỉ 99%.

```

[XGBoost] Thời gian Inference: 2.3078 giây
--- BÁO CÁO KẾT QUẢ ---

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2126118
1	0.96	0.99	0.97	673882
accuracy			0.99	2800000
macro avg	0.98	0.99	0.98	2800000
weighted avg	0.99	0.99	0.99	2800000

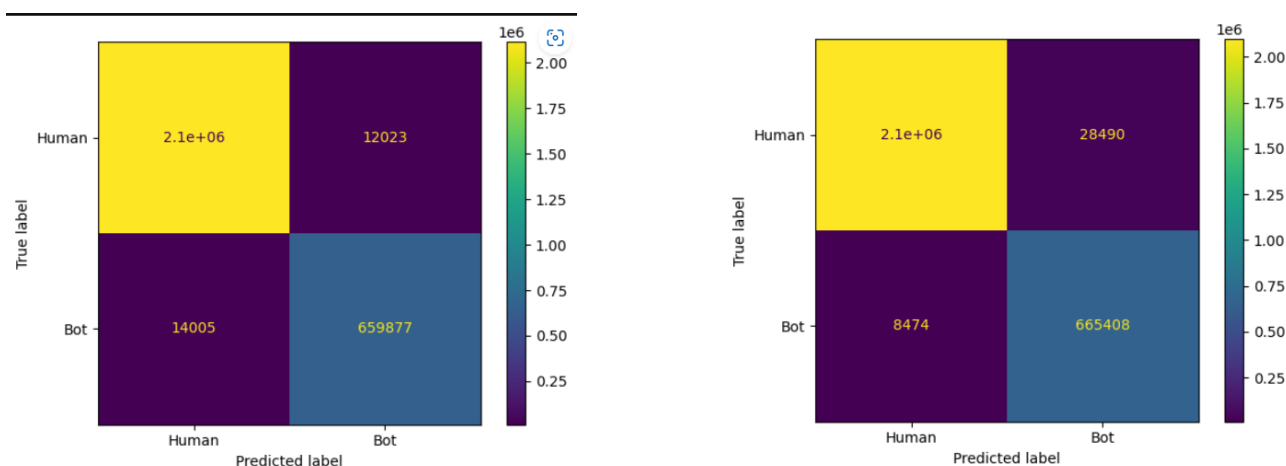
```

[Transformer] Thời gian Inference: 16.6050 giây
--- BÁO CÁO KẾT QUẢ ---

```

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	2126118
1.0	0.98	0.98	0.98	673882
accuracy			0.99	2800000
macro avg	0.99	0.99	0.99	2800000
weighted avg	0.99	0.99	0.99	2800000

Hình 18: Kết quả đạt được của hai mô hình trên tập dữ liệu huấn luyện chính



Hình 19: Ma trận nhầm lẫn (Confusion Matrix) của Transformer (trái) và XGBoost (phải)

- Từ ma trận nhầm lẫn của Transformer có thể thấy mô hình phân loại chính xác 2.114.095 người dùng hợp lệ, trong khi chỉ có 12.023 người dùng hợp lệ bị dự đoán nhầm là Bot (False Positive). Đối với lớp Bot, Transformer nhận diện đúng 659.877 mẫu và chỉ bỏ sót 14.005 mẫu (False Negative). Kết quả này cho thấy Transformer duy trì tỷ lệ False Positive rất thấp, góp phần hạn chế việc chặn nhầm người dùng hợp lệ. Đây là một yêu cầu quan trọng trong bài toán phát hiện Denial of Wallet, bởi việc từ chối các yêu cầu hợp lệ có thể ảnh hưởng trực tiếp đến trải nghiệm người dùng cũng như doanh thu của hệ thống dịch vụ.
- Đối với XGBoost, mô hình phân loại chính xác 2.097.628 người dùng hợp lệ và có 28.490 trường hợp bị nhận diện nhầm là Bot. Ở lớp Bot, XGBoost phát hiện chính xác 665.408 mẫu và chỉ bỏ sót 8.474 mẫu. Điều này giúp Recall của lớp Bot đạt xấp xỉ 99%, cho thấy mô hình có khả năng phát hiện gần như toàn bộ các mẫu tấn công trong tập kiểm thử. Đây là một đặc tính rất quan trọng trong các hệ thống phát hiện tấn công, nơi việc bỏ lọt đối tượng tấn công thường gây hậu quả nghiêm trọng hơn việc cảnh báo nhầm.
- Nhìn chung, cả XGBoost và Transformer đều đạt hiệu năng rất cao với độ chính xác tổng thể xấp xỉ 99%, đồng thời duy trì các chỉ số Precision,

Recall và F1-score ở mức từ 96% đến thậm chí là 100%. Tuy nhiên, hai mô hình thể hiện những ưu thế khác nhau. Transformer có tỷ lệ False Positive thấp hơn đáng kể (12.023 so với 28.490), nhờ đó giảm nguy cơ chặn nhầm người dùng hợp lệ và đạt sự cân bằng tốt giữa Precision và Recall. Ngược lại, XGBoost nhỉnh hơn về khả năng phát hiện Bot khi chỉ bỏ sót 8.474 mẫu, ít hơn Transformer 5.531 mẫu, đồng thời có thời gian suy luận chỉ 0,8242 ms một dòng dữ liệu (2,3078 giây với tập dữ liệu test có 2.800.000 mẫu), nhanh hơn khoảng 7 lần so với Transformer, 5,93 ms (16,6050 giây cho tập dữ liệu test 2.800.000 mẫu). Điều này cho thấy XGBoost phù hợp với các hệ thống yêu cầu phát hiện tấn công theo thời gian thực hoặc có giới hạn về tài nguyên tính toán, trong khi Transformer là lựa chọn phù hợp hơn khi mục tiêu là giảm thiểu các cảnh báo sai và hạn chế ảnh hưởng đến người dùng hợp lệ.

5.2. Kết luận

Kết quả thực nghiệm cho thấy cả XGBoost và Transformer đều có khả năng học được các đặc trưng phân biệt giữa lưu lượng hợp lệ và lưu lượng Bot trên bộ dữ liệu thử nghiệm. Mặc dù cả hai mô hình đều có thời gian suy luận ở mức mili giây, đáp ứng yêu cầu xử lý gần thời gian thực, lợi thế về tốc độ của XGBoost giúp mô hình phù hợp hơn với các hệ thống phải xử lý lượng lớn lưu lượng mạng liên tục, chẳng hạn như trong kịch bản tấn công Denial of Wallet của đề tài. Điều này cho thấy đối với bộ dữ liệu dạng bảng, các mô hình Gradient Boosting vẫn là một lựa chọn rất cạnh tranh so với các kiến trúc Deep Learning hiện đại.

Một phát hiện quan trọng của nghiên cứu là chất lượng và cấu trúc dữ liệu có ảnh hưởng lớn hơn lựa chọn mô hình. Trên bộ dữ liệu chứa Data Leakage, cả XGBoost và Transformer đều đạt kết quả gần như hoàn hảo nhưng không phản ánh đúng năng lực phát hiện tấn công thực tế.

Ngoài ra, việc cả hai mô hình đều đạt Accuracy và F1-score xấp xỉ 99% cho thấy bộ dữ liệu hiện tại có khả năng tồn tại các đặc trưng phân tách rất mạnh giữa hai lớp dữ liệu. Do đó, cần tiếp tục đánh giá khả năng tổng quát hóa của mô hình trên các bộ dữ liệu khác hoặc các kịch bản tấn công mới để xác định liệu hiệu năng cao này có thực sự phản ánh năng lực phát hiện tấn công trong môi trường thực tế hay không.

5.3. Hạn chế

Mặc dù các kết quả thực nghiệm cho thấy mô hình XGBoost và Transformer đạt hiệu năng cao trên tập dữ liệu được sử dụng, nghiên cứu của nhóm vẫn tồn tại một số hạn chế cần được xem xét.

Thứ nhất, phần lớn dữ liệu được sử dụng trong quá trình huấn luyện và đánh giá là dữ liệu mô phỏng. Mặc dù bộ dữ liệu này đã được nhóm tác giả kiểm chứng bằng nhiều phương pháp thống kê khác nhau, nó vẫn khó có thể phản ánh đầy đủ sự đa dạng và phức tạp của lưu lượng thực tế trong môi trường Serverless. Do đó, hiệu quả của mô hình trên dữ liệu thực tế vẫn cần được kiểm chứng thêm.

Thứ hai, nghiên cứu chưa xem xét ảnh hưởng của hiện tượng thay đổi phân phối dữ liệu theo thời gian. Trong môi trường vận hành thực tế, hành vi người dùng, lưu lượng hệ thống và kỹ thuật tấn công có thể thay đổi liên tục, dẫn đến nguy cơ suy giảm hiệu năng của mô hình sau một thời gian triển khai.

Cuối cùng, nghiên cứu mới dừng lại ở mức đánh giá ngoại tuyến trên tập dữ liệu có sẵn. Mô hình chưa được triển khai và kiểm thử trong môi trường Serverless thực tế để đánh giá các yếu tố như độ trễ suy luận, khả năng mở rộng, chi phí vận hành và tác động tới hệ thống đang hoạt động.

Chương 6. ĐỀ CƯƠNG KHÓA LUẬN

Nhóm đang có **hai** hướng đi cho *Khóa luận tốt nghiệp* trong tương lai:

6.1. Phát hiện Co-location Attack trong môi trường Serverless

- Trong đồ án chuyên ngành, nhóm đã nghiên cứu bài toán phát hiện tấn công Denial of Wallet (DoW) trong môi trường Serverless bằng các mô hình Machine Learning và Deep Learning. Kết quả thực nghiệm cho thấy cả XGBoost và Transformer đều đạt hiệu năng cao trên bộ dữ liệu thử nghiệm. Tuy nhiên, quá trình nghiên cứu cũng cho thấy một hạn chế lớn của lĩnh vực an ninh Serverless hiện nay là sự thiếu hụt các bộ dữ liệu thực tế phục vụ nghiên cứu. Phần lớn các công trình hiện nay đều phải sử dụng dữ liệu mô phỏng hoặc dữ liệu được tạo sinh bằng các công cụ chuyên biệt như DoWTS, dẫn đến khả năng mở rộng nghiên cứu sang các hình thức tấn công khác còn khá hạn chế.
- Từ thực tế đó, nhóm đề xuất hướng phát triển khóa luận tập trung vào bài toán phát hiện **Co-location Attack**, một trong những mối đe dọa đặc trưng của kiến trúc Serverless và điện toán đám mây nhiều khách hàng (Multi-tenant Cloud). Khác với Denial of Wallet chủ yếu khai thác chi phí vận hành, Co-location Attack tận dụng cơ chế lập lịch, tái sử dụng container và chia sẻ tài nguyên vật lý để đưa tiến trình của kẻ tấn công thực thi trên cùng hạ tầng với nạn nhân, từ đó tạo điều kiện cho nhiều dạng tấn công kênh kề (Side-channel Attack), đánh cắp thông tin hoặc ảnh hưởng tới hiệu năng dịch vụ.
- Để hiện thực hóa hướng nghiên cứu này, nhóm dự kiến kế thừa ý tưởng từ bài báo “*Kumo: A Security-Focused Serverless Cloud Simulator*” của giáo sư Wei Shao và các cộng sự. Kumo là một nền tảng mô phỏng Serverless được thiết kế chuyên biệt cho nghiên cứu an ninh, cho phép mô hình hóa chi tiết cơ chế lập lịch (Scheduler), vòng đời container, chính sách tái sử dụng container, phân bổ tài nguyên và nhiều kịch bản Co-location Attack trong các môi trường đa thuê. Không giống các công cụ kiểm thử hiệu năng truyền

thống chỉ tập trung tạo tải hệ thống, Kumo được xây dựng nhằm tái hiện các hành vi bảo mật ở mức hạ tầng, tạo điều kiện thuận lợi để sinh dữ liệu phục vụ nghiên cứu và đánh giá các cơ chế phòng vệ.

- Trong giai đoạn đầu của khóa luận, nhóm dự kiến nghiên cứu kiến trúc và mã nguồn của Kumo, từ đó hiệu chỉnh các tham số mô phỏng nhằm xây dựng nhiều kịch bản vận hành khác nhau. Các tham số có thể bao gồm số lượng máy chủ vật lý, số lượng container, thuật toán lập lịch, mức độ tái sử dụng container, mật độ yêu cầu, tỷ lệ chia sẻ tài nguyên, cùng nhiều cấu hình liên quan đến các chiến lược Co-location Attack. Việc thay đổi có hệ thống các tham số này sẽ cho phép tạo ra các bộ dữ liệu tổng hợp với quy mô lớn và đa dạng hơn so với các bộ dữ liệu hiện có, đồng thời phản ánh nhiều trạng thái vận hành khác nhau của môi trường Serverless.
- Trên cơ sở các bộ dữ liệu được sinh ra từ Kumo, nhóm sẽ kế thừa quy trình tiền xử lý dữ liệu, phân tích khám phá dữ liệu (EDA), trích xuất đặc trưng và xây dựng Pipeline học máy đã phát triển trong đề án chuyên ngành. Điều này giúp tận dụng những kết quả đã đạt được trước đó, đồng thời mở rộng đối tượng nghiên cứu từ bài toán phát hiện Denial of Wallet sang bài toán phát hiện Co-location Attack. Các mô hình Machine Learning và Deep Learning sẽ tiếp tục được nghiên cứu và đánh giá nhằm xác định khả năng nhận diện các dấu hiệu bất thường liên quan đến việc chia sẻ tài nguyên, hành vi lập lịch cũng như xác suất xảy ra hiện tượng đồng vị trí giữa tiến trình của kẻ tấn công và nạn nhân.
- Bên cạnh việc đánh giá hiệu năng của các mô hình phân loại, khóa luận cũng hướng tới việc phân tích mức độ ảnh hưởng của từng thành phần trong cơ chế vận hành Serverless đến nguy cơ xảy ra Co-location Attack. Việc này không chỉ giúp nâng cao khả năng giải thích của mô hình mà còn cung cấp các cơ sở định lượng phục vụ việc đề xuất các chính sách lập lịch hoặc cơ chế tái sử dụng container có tính bảo mật cao hơn.

- Về lâu dài, bộ dữ liệu tổng hợp được xây dựng từ Kumo có thể trở thành nền tảng cho nhiều hướng nghiên cứu tiếp theo như phát hiện bất thường trong môi trường Serverless bằng các mô hình Transformer, Graph Neural Network hoặc các mô hình học biểu diễn tiên tiến khác. Đồng thời, việc chủ động xây dựng được quy trình sinh dữ liệu cũng giúp giảm sự phụ thuộc vào các bộ dữ liệu công khai vốn còn rất hạn chế trong lĩnh vực an ninh Serverless, tạo tiền đề để phát triển các hệ thống phát hiện và phòng vệ có khả năng thích ứng với nhiều dạng tấn công mới trong tương lai.

6.2. Phát hiện bất thường trong môi trường Serverless bằng Dynamic Knowledge Graph và Dynamic GNN

- Một hướng phát triển khác đầy tiềm năng là khai thác tri thức ngữ nghĩa từ các nhật ký hệ thống (System Logs) để phục vụ cho việc phát hiện và dự đoán các mối đe dọa an ninh trong môi trường Serverless. So với các phương pháp học máy truyền thống chỉ khai thác dữ liệu dưới dạng bảng hoặc chuỗi thời gian, hướng tiếp cận dựa trên Knowledge Graph cho phép biểu diễn đầy đủ các thực thể trong hệ thống cùng các mối quan hệ giữa chúng, từ đó phản ánh chính xác hơn cấu trúc và hành vi của môi trường Serverless.
- Nhóm dự kiến kế thừa ý tưởng từ bài báo *“Observability and Incident Response in Managed Serverless Environments Using Ontology-Based Log Monitoring”* của giáo sư Lavi Ben-Shimol và các cộng sự. Trong nghiên cứu này, nhóm tác giả đã đề xuất một nền tảng giám sát và hỗ trợ ứng phó sự cố bằng cách xây dựng một Ontology tổng quát cho môi trường Serverless, sau đó chuyển đổi các nhật ký hệ thống thành Knowledge Graph nhằm biểu diễn mối quan hệ giữa người dùng, dịch vụ, hàm Serverless, tài nguyên, sự kiện và các thành phần khác trong hệ thống. Cách tiếp cận này giúp chuẩn hóa dữ liệu từ nhiều nguồn log khác nhau về cùng một mô hình ngữ nghĩa, tạo điều kiện thuận lợi cho việc phân tích và điều tra sự cố.

- Trên cơ sở đó, khóa luận dự kiến nghiên cứu việc mở rộng mô hình Knowledge Graph từ dạng tĩnh sang **Dynamic Knowledge Graph**, trong đó các nút và cạnh của đồ thị được cập nhật liên tục theo sự thay đổi của trạng thái hệ thống. Mỗi sự kiện phát sinh trong môi trường Serverless như việc kích hoạt hàm, khởi tạo hoặc tái sử dụng container, truy cập tài nguyên, giao tiếp giữa các dịch vụ hay các thay đổi về quyền truy cập đều được phản ánh thành các cập nhật của đồ thị theo thời gian. Nhờ đó, đồ thị không chỉ biểu diễn cấu trúc của hệ thống mà còn mô tả được tiến trình vận hành và sự biến đổi liên tục của môi trường Serverless.
- Sau khi xây dựng được Dynamic Knowledge Graph, nhóm dự kiến nghiên cứu và áp dụng các mô hình **Dynamic Graph Neural Network (Dynamic GNN)** nhằm học biểu diễn của đồ thị theo thời gian. Khác với các mô hình Graph Neural Network truyền thống chỉ xử lý đồ thị tĩnh, Dynamic GNN có khả năng khai thác đồng thời thông tin cấu trúc và diễn biến của các mối quan hệ giữa các thực thể. Điều này cho phép mô hình nhận diện những thay đổi bất thường trong quá trình vận hành của hệ thống, phát hiện các chuỗi sự kiện có nguy cơ dẫn đến tấn công hoặc sự cố bảo mật trước khi chúng gây ảnh hưởng đáng kể.
- Một lợi thế quan trọng của hướng nghiên cứu này là khả năng tận dụng đồng thời cả thông tin ngữ nghĩa và thông tin cấu trúc của hệ thống. Thay vì chỉ dựa vào các đặc trưng thống kê được trích xuất thủ công từ log, mô hình có thể học trực tiếp từ mạng lưới quan hệ giữa các thành phần trong môi trường Serverless. Điều này đặc biệt phù hợp với các cuộc tấn công có bản chất phức tạp như Data Leakage, Denial-of-Wallet, Co-location Attack hoặc các hình thức tấn công đa giai đoạn, nơi mà dấu hiệu bất thường không xuất hiện ở từng bản ghi riêng lẻ mà chỉ bộc lộ khi xem xét toàn bộ ngữ cảnh và mối liên kết giữa nhiều sự kiện khác nhau.
- Về lâu dài, hướng nghiên cứu này không chỉ dừng lại ở việc xây dựng mô hình phát hiện bất thường mà còn hướng tới việc hình thành một nền tảng

giám sát an ninh thông minh cho môi trường Serverless. Dynamic Knowledge Graph có thể đóng vai trò như một lớp biểu diễn tri thức thống nhất, trong khi Dynamic Graph Neural Network đảm nhiệm việc học các quy luật vận hành của hệ thống theo thời gian để dự đoán sớm các nguy cơ mất an toàn thông tin. Sự kết hợp giữa Ontology, Knowledge Graph và Graph Neural Network được kỳ vọng sẽ góp phần nâng cao khả năng giám sát, hỗ trợ ứng phó sự cố và tăng cường tính tự động cho các hệ thống bảo mật trong các nền tảng điện toán Serverless thế hệ mới.

TÀI LIỆU THAM KHẢO

- [1] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, V. Muthusamy, R. Rabbah, A. Slominski, and P. Suter, "Serverless Computing: Current Trends and Open Problems," in *Research Advances in Cloud Computing*, Springer, pp. 1–20, 2017, doi: 10.1007/978-981-13-5892-8_1.
- [2] Amazon Web Services, "AWS Shared Responsibility Model," AWS, [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.
- [4] D. Kelly, F. G. Glavin, and E. Barrett, "Denial of Wallet in Serverless Computing: A Systematic Literature Review," preprint/manuscript, 2025.
- [5] D. Mileski and H. Mihajloska, "Distributed Denial of Wallet Attack on Serverless Pay-as-you-go Model," in *Proceedings of the 30th Telecommunications Forum (TELFOR)*, Belgrade, Serbia, Nov. 2022, pp. 1–4, doi: 10.1109/TELFOR56187.2022.9983732.
- [6] D. Kelly, F. G. Glavin, and E. Barrett, "DoWTS – Denial-of-Wallet Test Simulator: Synthetic Data Generation for Preemptive Defence," *Journal of Intelligent Information Systems*, vol. 60, pp. 325–348, 2023, doi: 10.1007/s10844-022-00751-4.
- [7] J. M. Ortega Candel, F. J. Mora Gimeno, and H. Mora Mora, "Generation of a Dataset for DoW Attack Detection in Serverless Architectures," *Data in Brief*, vol. 52, Art. no. 109921, 2024, doi: 10.1016/j.dib.2023.109921.
- [8] J. M. Ortega Candel, F. J. Mora Gimeno, and H. Mora Mora, "Data Augmentation with Generative AI for DoW Attack Detection in Serverless Architectures," 2024.
- [9] A. Vaswani et al., "Attention Is All You Need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, 2017, doi: 10.48550/arXiv.1706.03762.
- [10] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

- [11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [12] J. M. Ortega Candel, F. J. Mora Gimeno, and H. Mora Mora, "Generation of a Dataset for DoW Attack Detection in Serverless Architectures - Dataset," *Mendeley Data*, vol. 1, 2024, doi: 10.17632/g8g9vdxyn.1.
- [13] J. M. Ortega Candel, F. J. Mora Gimeno, and H. Mora Mora, "Dataset for Denial-of-Wallet Attack Detection in Serverless Architectures," *Zenodo*, 2024, doi: 10.5281/zenodo.13758901.
- [14] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] A. Paszke et al., "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 5, 2023.
- [16] NVIDIA Corporation, *CUDA C Programming Guide*, NVIDIA, Santa Clara, CA, USA. [Online]. Available: <https://docs.nvidia.com/cuda/>
- [17] W. Shao, K. N. Khasawneh, S. Rafatirad, H. Homayoun, and C. Fang, "Kumo: A Security-Focused Serverless Cloud Simulator," *arXiv:2603.19787*, Mar. 2026, doi: 10.48550/arXiv.2603.19787.
- [18] L. Ben-Shimol, E. Grolman, A. Elyashar, I. Maimon, D. Mimran, O. Brodt, M. Strassmann, H. Lehmann, Y. Elovici, and A. Shabtai, "Observability and Incident Response in Managed Serverless Environments Using Ontology-Based Log Monitoring," *IEEE Transactions on Cloud Computing*, vol. 13, no. 4, pp. 1161–1176, 2025, doi: 10.1109/TCC.2025.3598060.
- [19] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," *arXiv:2006.10637*, 2020, doi: 10.48550/arXiv.2006.10637.